

操作系统习题集

目录

一、操作系统引论	2
二、进程管理	7
三、处理机管理	26
四、存储器管理	37
五、设备管理	52
六、文件管理	61
七、操作系统接口	69

一、操作系统引论

1. 设计现代 OS 的主要目标是什么？

答：（1）有效性 （2）方便性 （3）可扩充性 （4）开放性

2. 什么是操作系统？操作系统的基本特征是什么？并简单论述。

答：操作系统是计算机系统上的系统软件，是管理和控制计算机硬件和软件资源，合理组织计算机工作流程，以使用户有效利用计算资源。操作系统有四个特征：并发、共享、异步、虚拟，并发和共享是操作系统的最基本特征。

并发：是指两个或多个事件在同一时间间隔内发生。操作系统的并发性是指计算机系统中同时存在多个运行着的程序，因此它应该具有处理和调度多个程序同时执行的能力。

共享：是指系统中的资源（硬件资源和信息资源）可以被多个并发执行的程序共同使用，而不是被其中一个独占。资源共享有两种方式：互斥访问和同时访问。

异步：在多道程序环境下，允许多个程序并发执行，但由于资源有限，进程的执行不是一贯到底。而是走走停停，以不可预知的速度向前推进，这就是进程的异步性。异步性使得操作系统运行在一种随机的环境下，可能导致进程产生与时间有关的错误。但是只要运行环境相同，操作系统必须保证多次运行程序，都获得相同的结果。

虚拟：虚拟性是一种管理技术，把物理上的一个实体变成逻辑上的多个对应物，或把物理上的多个实体变成逻辑上的一个对应物的技术。采用虚拟技术的目的是为用户提供易于使用、方便高效的操作环境。

3. 什么是操作系统的基本功能？并简单论述。

答：操作系统的职能是管理和控制计算机系统上的所有硬、软件资源，合理地组织计算机工作流程，并为用户提供一个良好的工作环境和友好的接口。操作系统的基本功能包括：处理机管理、存储管理（内存管理）、设备管理、文件系统管理和用户接口等。

处理机管理：进程控制，进程同步，进程通信和调度。进程控制的主要任务是为作业创建进程，撤销已结束的进程，以及控制进程在运行过程中的状态转换。进程同步的主要任务是对诸进程的运行进行调节。进程通信的任务是实现在相互合作进程之间的信息交换。调度分为作业调度和进程调度。作业调度的基本任务是从后备队列中按照一定的算法，选择出若干个作业，为它们分配必要的资源；而进程调度的任务是从进程的就绪队列中，按照一定的算法选出一新进程，把处理机分配给它，并为它设置运行现场，是进程投入运行。

存储管理（内存管理）：内存分配，内存保护，地址映射和内存扩充等。内存分配的主要任务是为每道程序分配内存空间，提高存储器利用率，以减少不可用的内存空间，允许正在运行的程序申请附加的内存空间，以适应程序和数据动态增长的需要。内存保护的主要任务是确保每道用户程序都在自己的内存空间中运行，互不干扰。地址映射的主要任务是将地址空间中的逻辑地址转换为内存空间中与之对应的物理地址。内存扩充的主要任务是借助虚拟存储技术，从逻辑上去扩充内存容量。

设备管理：缓冲管理，设备分配和设备处理，以及虚拟设备等。完成用户提出的 I/O 请求，为用户分配 I/O 设备；提高 CPU 和 I/O 设备的利用率；提高 I/O 速度；以及方便用户使用 I/O 设备。

文件管理：对文件存储空间的管理，目录管理，文件的读，写管理以及文件的共享和保护。对用户文件和系统文件进行管理，以方便用户使用，并保证文件的安全性。

用户接口：为用户提供方便灵活地使用计算机的手段，一种是程序级接口，一种是作业级接口。

4. 操作系统各功能对应的物理设备分别有哪些？

答：处理机管理对应处理机，解决对处理机分配调度策略、分配实施和资源回收等问题；存储管理对应内存，主要工作是对内存储器进行分配、保护、扩充和管理；设备管理对应输入输出设备、通道、DMA 控制器等，对其进行分配和管理；文件管理对应软件资源，实现对软件资源逻辑结构、物理结构的管理和存储空间分配等。接口对用户，

为用户提供方便、灵活的使用计算机的手段。

5. 什么是批处理、分时和实时系统?各有什么特征?

答：批处理系统：操作员把用户提交的作业分类，把一批作业编成一个作业执行序列，由专门编制的监督程序(monitor)自动依次处理。

其主要特征是：用户脱机使用计算机、成批处理、多道程序运行。

分时系统：把处理机的运行时间分成很短的时间片，按时间片轮转的方式，把处理机分配给各进程使用。其主要特征是：交互性、多用户同时性、独立性。

实时系统：在被控对象允许时间范围内作出响应。其主要特征是：对实时信息分析处理速度要比进入系统快、要求安全可靠、资源利用率低。

6. 并行性和并发性的区别与联系。

答：

并行性和并发性是既相似又有区别的两个概念。并行性是指两个或多个事件在同一时刻发生。并发性是指两个或多个事件在同一时间间隔内发生。

在多道程序环境下，并发性是指在一段时间内，宏观上有多个程序在同时运行，但在单处理器系统中每一时刻却仅能有一道程序执行，故微观上这些程序只能是分时地交替执行。倘若在计算机系统中有多个处理器，则这些可以并发执行的程序便被分配到多个处理器上，实现并行执行，即利用每个处理器来处理一个可并发执行的程序。

7. 多道程序设计和多重处理有什么区别?

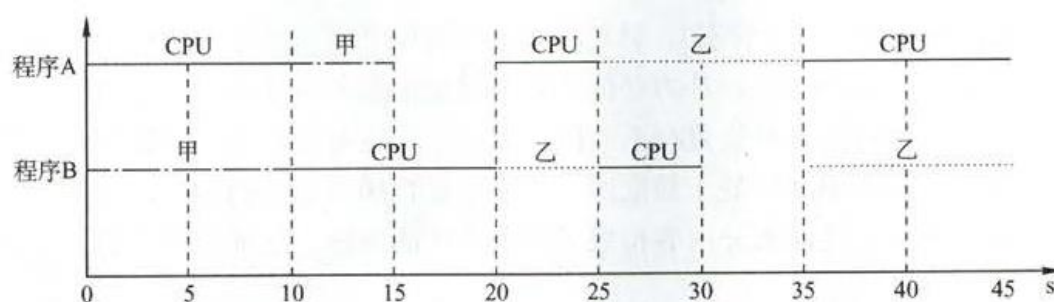
答：多道程序设计是作业之间自动调度执行、共享系统资源，并不是真正地同时执行多个作业；而采用多重处理的系统配置多个 CPU，能真正同时执行多道程序。要有效使用多重处理，必须采用多道程序设计技术，而多道程序设计原则上不一定要多重处理的支持。

8. 有两个程序，程序 A 依次使用 CPU 计 10s，使用设备甲计 5S，使用 CPU 计 5s，使用设备乙计 10s，使用 CPU 计 10s；程序 B 依次使用设备甲计 10S，使用 CPU 计 10s，使用设备乙计 5s，，使用 CPU 计 5s，使用设备乙计 10s。在单道程序环境下先执行 A 再执行 B，计算 CPU 的利用率是多少？在多道程序环境下，CPU 的利用率是多少？

答：

单道环境下, CPU 运行时间为 $(10+5+10)s+(10+5)s=40s$, 两个程序运行总时间为 $40s+40s=80s$, 故利用率是 $40/80=50\%$ 。

多道环境下, 运行情况如下图所示, CPU 运行时间为 $40s$, 两个程序运行总时间为 $45s$, 故利用率为 $40/45=88.9\%$ 。



9. 试从交互性、及时性以及可靠性方面, 将分时系统与实时系统进行比较。

答:(1)及时性:实时信息处理系统对实时性的要求与分时系统类似, 都是以人所能接受

的等待时间来确定; 而实时控制系统的及时性, 是以控制对象所要求的开始截止时间或完成截止时间来确定的, 一般为秒级到毫秒级, 甚至有的要低于 100 微妙。

(2)交互性:实时信息处理系统具有交互性, 但人与系统的交互仅限于访问系统中某些特定的专用服务程序。不像分时系统那样能向终端用户提供数据和资源共享等服务。

(3)可靠性:分时系统也要求系统可靠, 但相比之下, 实时系统则要求系统具有高度

的可靠性。因为任何差错都可能带来巨大的经济损失, 甚至是灾难性后果, 所以在实时系统中, 往往都采取了多级容错措施保障系统的安全性及数据的安全性。

10. 考察一个小的嵌入式操作系统或物联网系统, 写出系统引导和操作系统加载过程。

答:嵌入式系统的引导和操作系统加载过程分为两个二阶段。

第一阶段引导程序通常完成以下工作:

(1)硬件设备初始化。屏蔽所有中断, 设置 CPU 速度和时钟频率, RAM 初始化, 等等。

- (2)为第二阶段的引导程序加载准备 RAM 空间。初始化终端向量表。
- (3) 复制第二阶段的二进制代码到 RAM 空间中。
- (4) 设置好堆栈指针，为执行 C 语言代码做好准备。初始化堆栈。
- (5) 跳转到第二阶段。

第二阶段引导程序通常完成以下工作：

- (1) 其他硬件设备的初始化。
- (2) 检测系统内存映像。
- (3) 将操作系统内核映像及文件系统映像从 Flash 读取到系统 RAM 中。
- (4) 为操作系统内核设置启动参数
- (5) 调用操作系统内核

11. 操作系统的发展历程

- (1) 最初是手工操作阶段，需要人工干预，有严重的缺点，此时尚未形成操作系统
- (2) 早期批处理分为联机和脱机两类，其主要区别在与 I/O 是否受主机控制
- (3) 多道批处理系统中允许多道程序并发执行，与单道批处理系统相比有质的飞跃

二、进程管理

1. 画出下面四条语句的前趋图：

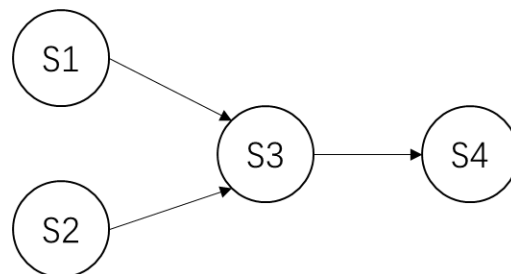
S1: $a=x+y$;

S2: $b=z+1$;

S3: $c=a-b$;

S4: $w=c+1$;

答：其前趋图为：



2. 举例说明程序并发执行时为什么会失去封闭性和可再现性？

答：因为程序并发执行时，多个程序共享系统中的各种资源，资源状态需要多个程序来改变，即存在资源共享性使程序失去封闭性；而失去了封闭性导致程序失去可再现性。例如：

程序1	程序2
...	...
$R1=X$	$R2=X$
$R1=R1+1$	$R2=R2+1$
$X=R1$	$X=R2$
...	...

执行顺序1：程序1→程序2
结果为：X增加2

执行顺序2：... $R1=X$; $R2=X$; $R1=R1+1$; $R2=R2+1$; $X=R1$; $X=R2$
结果为：X增加1

... 还可有许多其它组合...

3. 论述进程和程序的区别与联系。

答：

(1) 动态性是进程最基本的特性，进程是一个动态概念，而程序是一个静态概念，程序是指令的有序集合，无执行含义，是静态实体，进程则强调执行的过程。

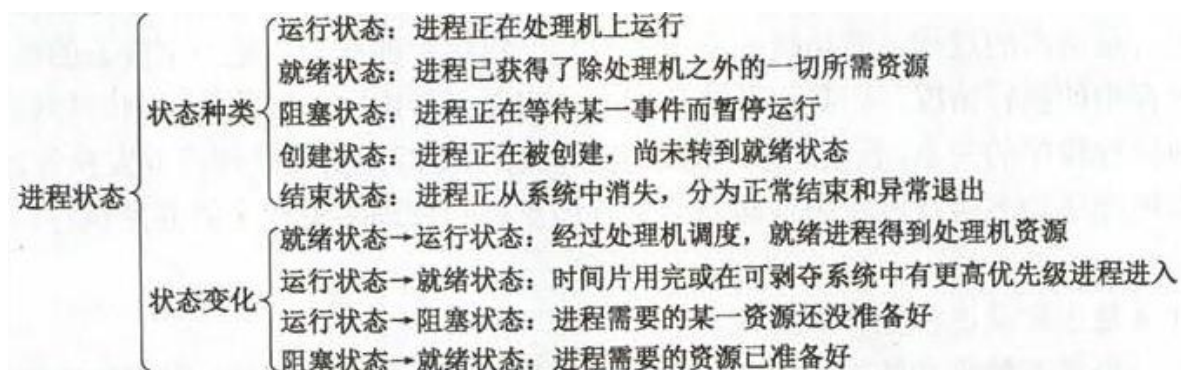
(2) 并发性是进程的重要特征，引入进程的目正是为了使其程序能和其它进程的程序并发执行，而程序是不能并发执行的。

(3) 独立性是指进程实体是一个能独立运行的基本单位，同时也是系统中独立获得资源和独立调度的基本单位。而对于未建立任何进程的程，都不能作为一个独立的单位参加运行。

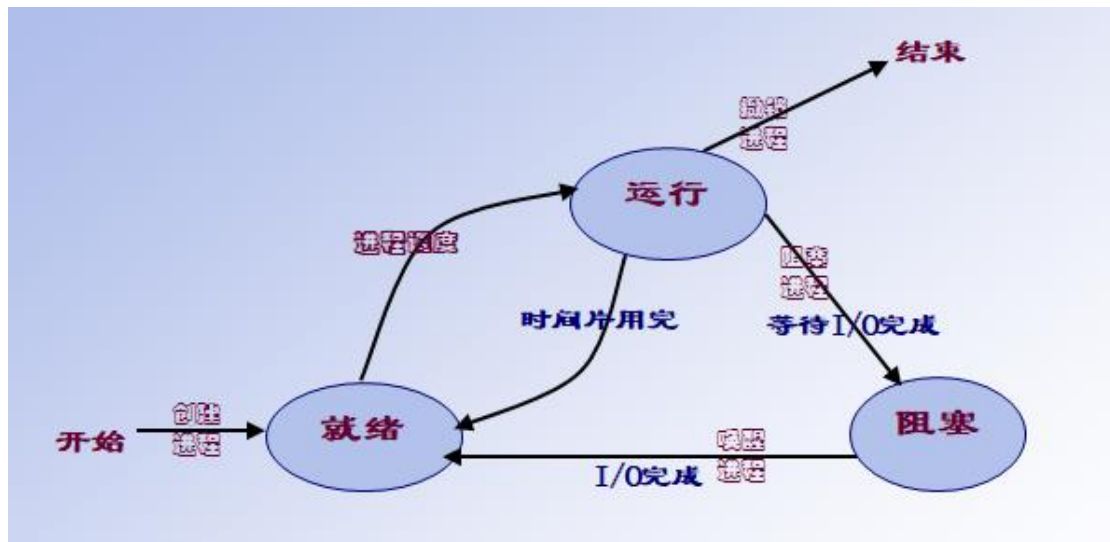
(4) 不同的进程可以包含同一个程序，同一程序在执行中也可以产生多个进程。

4. 进程有几种状态？并说明进程在不同状态之间转换的过程和原因。

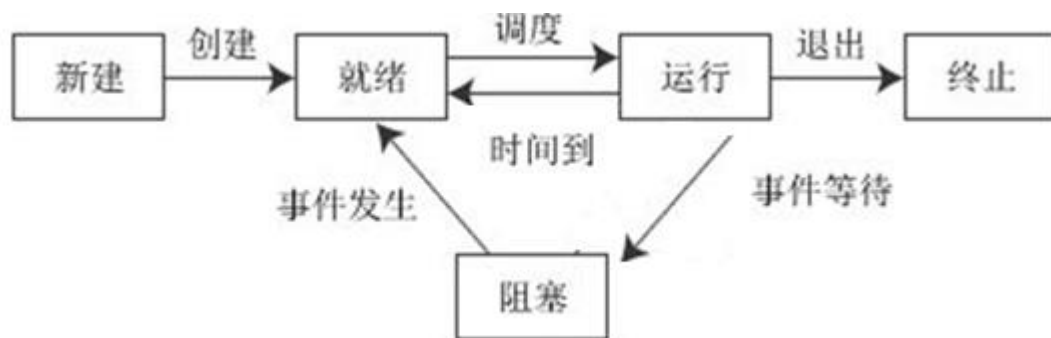
答：基本为三态（运行、就绪、阻塞）或五态（运行、就绪、阻塞、创建、结束）



三态图

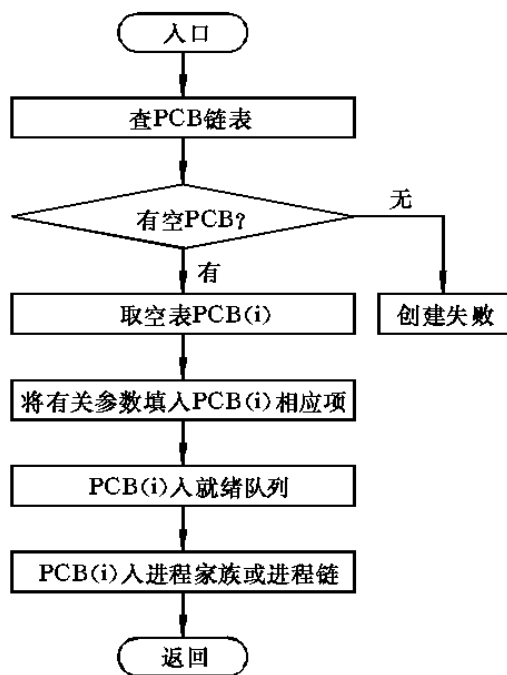


五态图

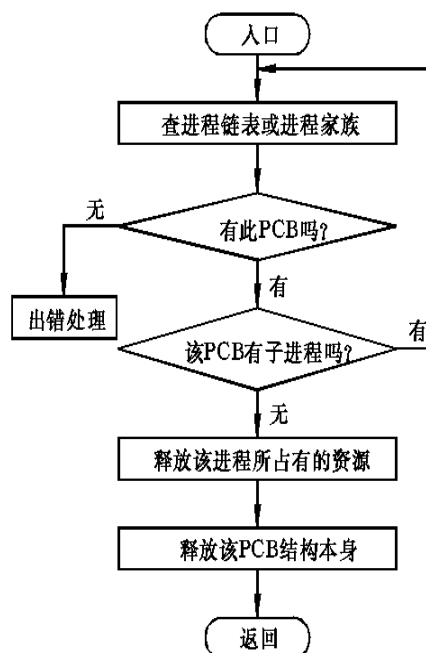


5. 什么是原语？进程控制原语都有哪些，并简要说明。

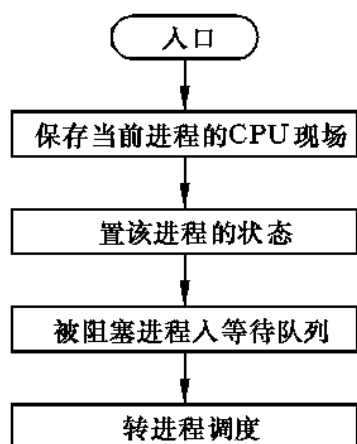
答：原语是指由若干条指令组成的，用于完成某一特定功能的一段程序，具有不可分割性，即原语的执行必须是连续的，在执行过程中不允许被中断。创建原语、撤销原语、阻塞原语、唤醒原语。



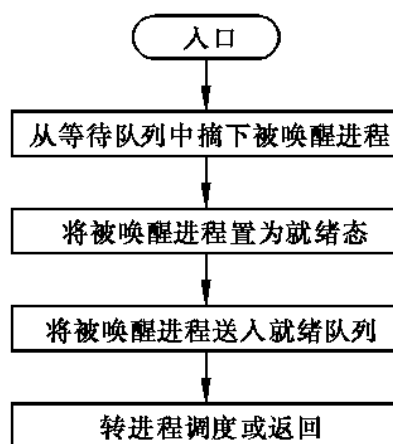
创建原语



撤消原语



阻塞原语图



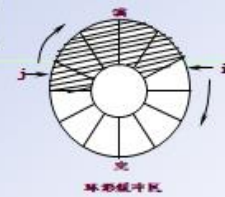
唤醒原语

6. 什么是进程间的互斥?什么是进程间同步?使用 P, V 操作举例说明互斥和同步的实现。【注: P 即 wait,V 即 signal】

答: 进程间的互斥是指: 一组并发进程中的一个或多个程序段, 因共享某一公有资源而导致它们必须以一个不许交叉执行的单位执行, 即不允许两个以上的共享该资源的并发进程同时进入临界区。

进程间的同步是指: 异步环境下的一组并发进程因直接制约互相发送消息而进行互相合作、互相等待, 各进程按一定的速度执行的过程。

生产者/消费者算法描述1



```

var mutex, empty, full: psemaphore;
i, j, goods: integer; buffer: array
[0...n-1] of item;
procedure producer; 生产者进程
begin
  while true do
  begin
    produce next product;
    P(empty);
    P(mutex);
    buffer(i):=product;
    i:=(i+1) mod(n);
    V(mutex);
    V(full);
  end
end;
    
```

```

procedure consumer; 消费者进程
begin
  while true do
  begin
    P(full);
    P(mutex);
    goods:= buffer(j);
    j:=(j+1) mod(n);
    V(mutex);
    V(empty);
    consume product;
  end
end;
    
```

7. 设有 5 个哲学家，共享一张放有五把椅子的桌子，每人分得一把椅子。但是，桌子上总共只有 5 支筷子，在每人两边分开各放一支。哲学家们在肚子饥饿时才试图分两次从两边拾起筷子就餐。

条件：

- (1) 只有拿到两支筷子时，哲学家才能吃饭。
- (2) 如果筷子已在他人手上，则该哲学家必须等待到他人吃完之后才能拿到筷子。
- (3) 任一哲学家在自己未拿到两支筷子吃饭之前，决不放下自己手中的筷子。

试解答以下问题：

- (1) 描述一个保证不会出现两个邻座同时要求吃饭的通信算法。
- (2) 描述一个既没有两邻座同时吃饭，又没有人饿死（永远拿不到筷子）的算法。在什么情况下，5 个哲学家全部吃不上饭？

答：【注：P 即 wait, V 即 signal】

- (1)、设信号量 $c[0] \sim c[4]$, 初始值均为 1, 分别表示 i 号筷子被拿 ($i=0, 1, 2, 3, 4$),

```

send(i):第 i 个哲学家要吃饭
begin
P(c[i]);
P(c[i+1 mod 5]);
eat;
V(c[i+1 mod 5]);
V(c[i]);
End;

```

该过程能保证两邻座不同时吃饭,但会出现 5 个哲学家一人拿一只筷子,谁也吃不上饭的死锁情况.

(2)、解决思路如下:让奇数号的哲学家先取右手边的筷子,让偶数号的哲学家先取左手边的筷子.这样,任何一个哲学家拿到一只筷子之后,就已经阻止了他邻座的一个哲学家吃饭的企图,除非某个哲学家一直吃下去,否则不会有人饿死.

```

send(i):第 i 个哲学家要吃饭
begin
    if i mod 2==0 then
    {
        P(c[i]);
        P(c[i+1 ]mod 5);
        eat;
        V(c[i]);
        c[i+1 mod 5]);
    }
    else
    {
        P(c[i+1 mod 5]);
        P(c[i]);
        Eat;
        V(c[i+1 mod 5]);
    }
end

```

```

        V(c[i]);
    }
end

```

8. 面包师有很多面包，由 n 个销售人员推销。每个顾客进店后去一个号，并且等待叫号，当一个销售人员空闲下来时，就叫下一个号。试设计一个销售人员和顾客同步算法。【注：P 即 wait, V 即 signal】

顾客进店后按序取号，并等待叫号；销售人员空闲之后也是按序叫号，并销售面包。因此同步算法只要对顾客取号和销售人员叫号进行合理同步即可。我们使用两个变量 i 和 j 分别记录当前的取号值和叫号值，并各自使用一个互斥信号量用于对 i 和 j 进行访问和修改。

```

int i=0,j=0;
semaphore mutex_i=1,mutex_j=1;
Consumer() {                                //顾客
    进入面包店;
    P(mutex_i);                             //互斥访问 i
    取号 i;
    i++;
    V(mutex_i);                             //释放对 i 的访问
    等待叫号 i 并购买面包;
}
Seller() {                                  //销售人员
    while(1){
        P(mutex_j);                         //互斥访问 j
        if(j<i){                            //号 j 已有顾客取走并等待
            叫号 j;
            j++;
            V(mutex_j);                     //释放对 j 的访问
            销售面包;
        }
        else{                               //暂时没有顾客在等待
            V(mutex_j);                     //释放对 j 的访问
            休息片刻;
        }
    }
}

```

9.

某工厂有两个生产车间和一个装配车间，两个生产车间分别生产 A、B 两种零件，装配车间的任务是把 A、B 两种零件组装成产品。两个生产车间每生产一个零件后都要分别把它们送到专配车间的货架 F1、F2 上。F1 存放零件 A，F2 存放零件 B，F1 和 F2 的容量均可以存放 10 个零件。装配工人每次从货架上取一个零件 A 和一个零件 B 后组装成产品。请用 P、V 操作进行正确管理。

。解答：

本题是生产者-消费者问题的变形，生产者“车间 A”和消费者“装配车间”共享缓冲区“货架 F1”；生产者“车间 B”和消费者“装配车间”共享缓冲区“货架 F2”。因此，可为它们设置 6 个信号量，其中，empty1 对应货架 F1 上的空闲空间，其初值为 10；full1 对应货架 F1 上面的 A 产品，其初值为 0；empty2 对应货架 F2 上的空闲空间，其初值为 10；full2 对应货架 F2 上面的 B 产品，其初值为 0；mutex1 用于互斥地访问货架 F1，其初值为 1；mutex2 用于互斥地访问货架 F2，其初值为 1。

A 车间的工作过程可描述为：

```
while(1){
    生产一个产品 A;
    P(empty1);                //判断货架 F1 是否有空
    P(mutex1);                //互斥访问货架 F1
    将产品 A 存放到货架 F1 上;
    V(mutex1);                //释放货架 F1
    V(full1);                  //货架 F1 上的零件 A 的个数加 1
}
```

B 车间的工作过程可描述为：

```
while(1){
    生产一个产品 B;
    P(empty2);                //判断货架 F2 是否有空
    P(mutex2);                //互斥访问货架 F2
    将产品 B 存放到货架 F2 上;
    V(mutex2);                //释放货架 F2
    V(full2);                  //货架 F2 上的零件 B 的个数加 1
}
```

装配车间的工作过程可描述为：

```
while(1){
    P(full1);                  //判断货架 F1 上是否有产品 A
    P(mutex1);                //互斥访问货架 F1
    从货架 F1 上取一个 A 产品;
    V(mutex1);                //释放货架 F1
    V(empty1);                 //货架 F1 上的空闲空间数加 1
    P(full2);                  //判断货架 F2 上是否有产品 B
    P(mutex2);                //互斥访问货架 F2
```

```
    从货架 F2 上取一个 B 产品;
    V(mutex2);                //释放货架 F2
    V(empty2);                 //货架 F2 上的空闲空间数加 1
    将取得的 A 产品和 B 产品组装成产品;
}
```

【注：P 即 wait,V 即 signal】

10.

有 A、B 两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题。将答案和向对方提出的新问题组成一个邮件放入对方的邮箱中。假设 A 的信箱最多放 M 个邮件，B 的信箱最多放 N 个邮件。初始时 A 的信箱中有 x 个邮件 ($0 < x < M$)，B 的信箱中有 y 个 ($0 < y < N$)。辩论者每取出一个邮件，邮件数减 1。A 和 B 两人的操作过程描述如下：

CoBegin

<pre> A{ while(TRUE){ 从 A 的信箱中取出一个邮件; 回答问题并提出一个新问题; 将新邮件放入 B 的信箱; } } </pre>	<pre> B{ while(TRUE){ 从 B 的信箱中取出一个邮件; 回答问题并提出一个新问题; 将新邮件放入 A 的信箱; } } </pre>
--	--

CoEnd

当信箱不为空时，辩论者才能从信箱中取邮件，否则等待。当信箱不满时，辩论者才能将新邮件放入信箱，否则等待。请添加必要的信号量和 P、V（或 wait、signal）操作，以实现上述过程的同步。要求写出完整过程，并说明信号量的含义和初值。

答：

```

semaphore Full_A = x;    //Full_A 表示 A 的信箱中的邮件数量
semaphore Empty_A = M-x; //Empty_A 表示 A 的信箱中还可存放的邮件数量
semaphore Full_B = y;    //Full_B 表示 B 的信箱中的邮件数量
semaphore Empty_B = N-y; //Empty_B 表示 B 的信箱中还可存放的邮件数量
semaphore mutex_A = 1;   //mutex_A 用于 A 的信箱互斥
semaphore mutex_B = 1;   //mutex_B 用于 B 的信箱互斥

```

Cobegin

<pre> A{ while(TRUE){ P(Full_A); P(mutex_A); 从 A 的信箱中取出一个邮件; V(mutex_A); V(Empty_A); 回答问题并提出一个新问题; P(Empty_B); P(mutex_B); 将新邮件放入 B 的信箱; V(mutex_B); V(Full_B); } } </pre>	<pre> B{ while(TRUE){ P(Full_B); P(mutex_B); 从 B 的信箱中取出一个邮件; V(mutex_B); V(Empty_B); 回答问题并提出一个新问题; P(Empty_A); P(mutex_A); 将新邮件放入 A 的信箱; V(mutex_A); V(Full_A); } } </pre>
--	--

【注：P 即 wait, V 即 signal】

11. 什么是线程?试述线程与进程的区别。

答：线程是在进程内用于调度和占有处理机的基本单位，它由线程控制表、存储线程上下文的用户栈以及核心栈组成。线程可分为用户级线程、核心级线程以及用户 / 核心混合型线程等类型。其中用户级线程在用户态下执行，CPU 调度算法和各线程优先级都由用户设置，与操作系统内核无关。核心级线程的调度算法及线程优先级的控制权在操作系统内核。混合型线程的控制权则在用户和操作系统内核二者。线程与进程的主要区别有：

(1) 进程是资源管理的基本单位，它拥有自己的地址空间和各种资源，例如内存空间、外部设备等；线程只是处理机调度的基本单位，它只和其他线程一起共享进程资源，但自己没有任何资源。

(2) 以进程为单位进行处理机切换和调度时，由于涉及到资源转移以及现场保护等问题，将导致处理机切换时间变长，资源利用率降低。以线程为单位进行处理机切换和调度时，由于不发生资源变化，特别是地址空间的变化，处理机切换的时间较短，从而处理机效率也较高。

(3) 对用户来说，多线程可减少用户的等待时间。提高系统的响应速度。例如，当一个进程需要对两个不同的服务器进行远程过程调用时，对于无线程系统的操作系统来说需要顺序等待两个不同调用返回结果后才能继续执行，且在等待中容易发生进程调度。对于多线程系统而言，则可以在同一进程中使用不同的线程同时进行远程过程调用，从而缩短进程的等待时间。

(4) 线程和进程一样，都有自己的状态。也有相应的同步机制，不过，由于线程没有单独的数据和程序空间，因此，线程不能像进程的数据与程序那样，交换到外存存储空间。从而线程没有挂起状态。

(5) 进程的调度、同步等控制大多由操作系统内核完成，而线程的控制既可以由操作系统内核进行，也可以由用户控制进行。

12. 在创建一个进程时所要完成的主要工作是什么？

答：

- (1) OS 发现请求创建新进程事件后，调用进程创建原语 `Creat()` ；

- (2) 申请空白 PCB ；
- (3) 为新进程分配资源；
- (4) 初始化进程控制块；
- (5) 将新进程插入就绪队列 。

13. 在撤销一个进程时所完成的主要工作是什么？

答：

- (1) 根据被终止进程标识符，从 PCB 集中检索出进程 PCB ，读出该进程状态。
- (2) 若被终止进程处于执行状态，立即终止该进程的执行，置调度标志真，指示该进程被终止后重新调度。
- (3) 若该进程还有子进程，应将所有子孙进程终止，以防它们成为不可控进程。
- (4) 将被终止进程拥有的全部资源，归还给父进程，或归还给系统。
- (5) 将被终止进程 PCB 从所在队列或列表中移出，等待其它程序搜集信息。

14. 在控制系统中的数据采集任务，把所采集的数据送一单缓冲区；计算任务从该单缓冲中取出数据进行计算，试写出利用信号量机制实现两者共享单缓冲的同步算法。

答：

A:

```

Var mutex, empty, full: semaphore:=1, 1, 0;
gather:
begin repeat
    gather data in nextp;
    wait(empty);
    wait(mutex);
    buffer:=nextp;
    signal(mutex);
    signal(full);
until false;
end

```

```

compute:
begin repeat
    wait(full);
    wait(mutex);
    nextc:=buffer;
    signal(mutex);
    signal(empty);
    compute data in nextc;
until false;
end

```

```

B:
Var empty, full: semaphore:=1, 0;
gather:
begin repeat
    gather data in nextp;
    wait(empty);
    buffer:=nextp;
    signal(full);
until false;
end

```

```

compute:
begin repeat
    wait(full);
    nextc:=buffer;
    signal(empty);
    compute data in nextc;
until false;
end

```

15. 为什么要在 OS 中引入线程？

答：在操作系统中引入线程，则是为了减少程序在并发执行时所付出的时空开销，使 OS 具有更好的并发性，提高 CPU 的利用率。进程是分配资源的基本单位，而线程则是系统调度的基本单位。

16. 设有一台计算机，有两个 I/O 通道，分别接一台卡片输入机和一台打印机。卡片机把一叠卡片逐一输入到缓冲区 B1 中，加工处理后再搬到缓冲区 B2 中，并在打印机上印出，问：

- 1) 系统要设几个进程来完成这个任务？各自的工作是什么？
- 2) 这些进程间有什么样的相互制约关系？
- 3) 用 P、V 操作写出这些进程的同步算法。

答：1) 系统可设三个进程来完成该任务：Read 进程负责从卡片输入机上读入卡片信息，输入到缓冲区 B1 中；Get 进程负责从缓冲区 B1 中取出信息，进行加工处理，之后将结果送到缓冲区 B2 中；Print 进程负责从缓冲区 B2 中取出信息，并在打印机上打印输出。

2) 操作过程： Read 进程受 Get 进程的影响，B1 缓冲区中放满信息后 Read 进程要等待 get 进程将其中信息全部取走后才能读入信息； Get 进程受 Read 进程和 Print 进程的约束：B1 缓冲区中信息放满后，Get 进程才可从中取走信息，且 B2 缓冲区信息被取空后 Get 进程才能将加工结果送入其中； Print 进程受 Get 进程的约束，B2 缓冲区中信息放满后 Print 进程方可取出信息进行打印输出。

3) 信号量的含义及初值：

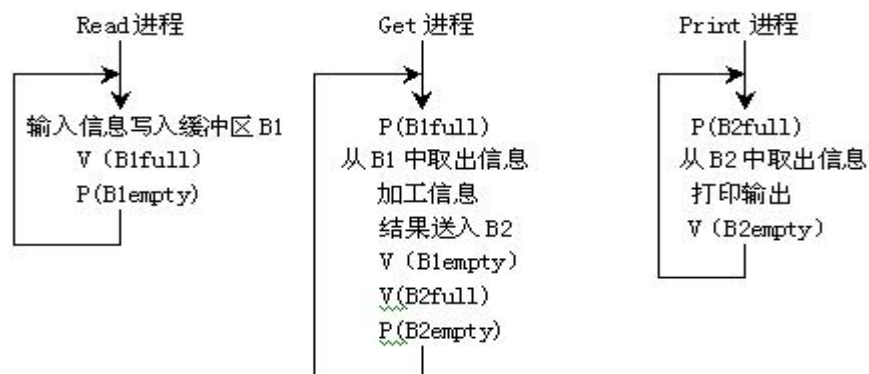
B1full——缓冲区 B1 满，初值为 0

B1empty——缓冲区 B1 空，初值为 0

B2full——缓冲区 B2 满，初值为 0

B2empty——缓冲区 B2 空，初值为 0

操作框图如下：



代码：略

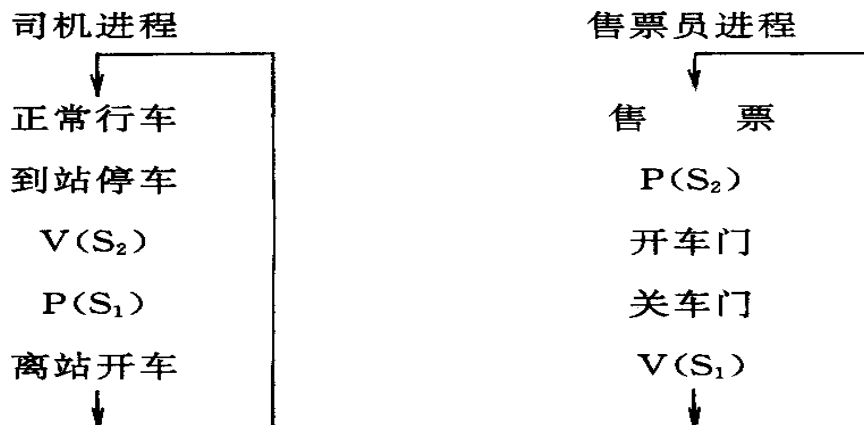
17. 用信号量实现司机和售票员的同步。

设 S_1 为司机的私用信号量，0 表不许开车，1 允许开车，初值为 0

S_2 为售票员的私用信号量，0 表不许开门，1 允许开门，初值为 0

由于初始状态是汽车行车和售票员售票。所以初值都为 0

答：则司机和售票员的同步过程描述如下：



代码：略

18. 桌子上有一只盘子，每次只能放入一只水果，爸爸专向盘子中放苹果，妈妈专向盘子中放桔子，一个儿子专等吃盘子中的桔子，一个女儿专等吃盘子里的苹果。只有盘子空则爸爸或妈妈就可向盘子中放一只水果，仅当盘子中有自己需要的水果时，儿子或女儿可从盘子中取出。

把爸爸、妈妈、儿子、女儿看作四个进程，用 PV 操作进行管理，使这四个进程能正确的并发执行。

答：爸爸和妈妈存放水果时必须互斥。临界资源为盘子

儿子和女儿分别吃桔子和苹果。

爸爸放了苹果后，应把“盘中有苹果”的消息发送给女儿；

妈妈放了桔子后，应把“盘中有桔子”的消息发送给儿子；

取走果品后应该发送“盘子可放水果”的消息，但不特定发给爸爸或妈妈，应该通过竞争资源(盘子)的使用权来决定

S 是否允许向盘子中放入水果，初值为 1，表示允许放入，且只允许放入一只。

SP 表示盘子中是否有苹果，初值为 0，表示盘子为空，不许取，SP=1 时可以取。

SO 表示盘子中是否有桔子，初值为 0，表示盘子为空，不许取，SP=1 时可以取。

至于儿子或女儿取走水果后要发送“盘子中可存放水果”的消息，只要调用 V(S)就可达到目的，不必在增加信号量了。

Begain

S, SP, SO: semaphore

S:=1; SP:=0; SO:=0;

Cobegain

process father

begain

L 1:have an apple;

P(S);

put an apple;

V(SP);

go to L 1

end;

process mother

begain

L 2:have an orange;

P(S);

put an orange;

V(SO);

go to L 2

end;

process son

begain

L3: P(SO);

get an orange;

V(S);

eat an orange;

go to L 3

end;


```

process   daught
begin
    L4: P(SP);
        get  an apple;
        V(S);
        eat an apple;
        go to L4
    end ;
coend;
end ;

```

19. 原子操作与临界区的区别是什么？

答：原子操作是由处理器保证的不可分割的操作。对于简单的原子操作，处理器会提供单条指令以保证原子性；对于复杂的原子操作，通过天机自旋锁（spinlock）来实现执行过程中不被打断。

临界区是不允许多个并发进程交叉执行的一段程序。临界区的实现方式是：在临界区设置和检查访问标志。进程访问临界区时，通过访问标志判断临界区是否有其他进程在访问。如果有，则等待并循环检查；如果没有，则进入临界区。进程离开临界区时重置访问标志。

20、简述 P、V 源于和加锁法实现进程间互斥的区别。

答：互斥的加锁实现原理如下。当某个进程进入临界区之后，它将锁上临界区，直到它退出临界区时为止。并发进程在申请进入临界区时，首先测试该临界区是否是上锁的。如果该临界区已被锁住，则并发进程要等到该临界区开锁之后才有可能获得临界区。

但是加锁法存在如下弊端：①循环测试锁定位将损耗较多的 CPU 计算时间；②产生不公平现象。

为此，P、V 原语法采用信号量管理相应临界区的公有资源，信号量的数值仅能由 P、V 原语操作改变，而 P、V 原语执行期间不允许中断发生。其过程是：当某个进程正在临界区内执行时，其他进程如果执行了 P 原语，则该进程并不像执行 lock 时那样因进不了临界区

而返回 lock 的起点，等以后重新执行测试，而是在等待队列中等待由其他进程执行 V 原语操作释放资源后再进入临界区，这时 P 原语才算真正结束。若有多个进程执行 P 原语操作而进入等待状态，一旦有进程执行 V 原语操作释放资源，则等待进程中的一个进入临界区，其余的进程继续等待。

总之，加锁法是采用反复测试 lock 实现互斥的，存在 CPU 浪费和不公平现象；而 P、V 原语使用了信号量，克服了加锁法的弊端。

21、编写一个程序，使用系统调用 fork 生成 3 个子程序，并使用系统调用 pipe 创建一条管道，使得这 3 个子进程和父进程公用统一管道进行通信

```

main()
{
    int r,p1,p2,p3,fd[2];                /* fd[2]为管道文件读写标识 */
    char buf[50],s[5];
    pipe(fd);                            /* 创建管道 */
    while((p1=fork())!=-1);              /* 创建子进程 1 */
    if(p1==0)                            /* 在子进程 1 中执行 */
    {
        lockf(fd[1],1,0);                /* 锁定写过程 */
        sprintf(buf,"child process P1 is sending message!\n");
        printf("child process P1!\n");
        write(fd[1],buf,50);             /* 将 buf 中的数据写入管道 */
        sleep(5);                        /* 睡眠,等待父进程读出 */
        lockf(fd[1],0,0);                /* 解锁 */
        exit(0);
    }
    else
    {
        while((p2=fork())!=-1);          /* 创建子进程 2 */
        if(p2==0)                        /* 在子进程 2 中执行 */
        {
            lockf(fd[1],1,0);            /* 锁定写过程 */
            sprintf(buf, "child process P2 is sending message!\n");
            /* 数据写入 buf */
            printf("child process P2!\n");
            write(fd[1],buf,50);          /* 将 buf 中的数据写入管道 */
            sleep(5);                    /* 同步等待父进程读 */
            lockf(fd[1],0,0);            /* 解锁 */
        }
    }
}

```

```

        exit(0);                                /* 释放进程资源 */
    }
    else
    {
        while((p3=fork())!=-1);                /* 创建子进程 3 */
        if(p3==0)                               /* 在子进程 3 中执行 */
        {
            lock(fd[1],1,0);
            sprintf(buf,"child process P3 is sending message!\n");
            printf("child process P3!\n");
            write(fd[1],buf,50);
            sleep(5);
            lockf(fd[1],0,0);
            exit(0);
        }
        wait(0);                                /* 父进程等待子进程先执行 */
        if(r=read(fd[0],s,50)==-1)              /* 读管道中的内容到 s 中 */
            printf("can't read pipe\n");
        else
            printf("%s\n",s);
        wait(0);                                /* 等待另一个子进程执行 */
        if(r=read(fd[0],s,50)==-1)
            printf("can't read pipe\n");
        else
            printf("%s\n",s);
        wait(0);                                /* 等待最后一个子进程执行 */
        if(r=read(fd[0],s,50)==-1)
            printf("can't read pipe\n");
        else
            printf("%s\n",s);
        exit(0);
    }
}
}

```

22、假设系统有四类资源：磁带驱动器、绘图仪、打印机和卡片穿孔机。各类资源的总数用 $W=(6, 3, 4, 2)$ 表示，即有 6 台磁带驱动器，3 台绘图仪，4 台打印机，2 台卡片穿孔机。

现有五个进程 A、B、C、D 和 E，已获得的资源的种类及数量如下所示：

括号外面的数字代表：已获得的资源的种类及数量

进程	磁带驱动器	绘图仪	打印机	穿孔机
A	3 (1)	0 (1)	1 (0)	1 (0)
B	0 (0)	1 (3)	0 (1)	0 (2)
C	1 (5)	1 (1)	1 (0)	0 (0)
D	1 (0)	1 (0)	0 (1)	1 (0)

E	0 (2)	0 (1)	0 (1)	0 (0)
---	-------	-------	-------	-------

括号里面的数字代表：尚需资源的种类及数量

(1) 请找出一个执行的安全队列。

(2) 如果 B 请求 (0110)，能否分配给它？如分配给它会否死锁？

(3) 如果 C 请求 (1000)，能否分配给它？如分配给它会否死锁？

(4) 如果 E 请求 (1020)，能否分配给它？如分配给它会否死锁？

答：结合课件以及 23 题自行解答

23. 银行家算法题：若出现下述的资源分配情况：

Process	Current- Allocation	Still-Need	Available
P0	0032	0012	2022
P1	1000	1130	
P2	1301	1310	
P3	0301	0032	
P4	0012	1023	

该状态是否安全？若安全，则列出一个安全序列。

(2) 如果进程 P2 提出请求 Request(1、0、1、0) 后，
系统能否将资源分配给它，写出分析过程（文字描述）

解：(1) 该状态使安全的。P0 p3 p4 p1 p2 是它的一个安全序列。

(2) 假设对进程 p2 提出请求 Reqrst (1、0、1、0。) 予以满足，则系统资源剩余量为 1、0、1、2。此时资源申请可以满足的进程只有 p3 和 p4，假设先让 p3 完成，它完成后系统资源剩余量为 1、3、4、5。此时资源申请可以满足的进程有 p1p2p4，假设先让 p4 完成，它完成后系统资源剩余量为 1、3、5、7。此时资源申请可以满足的进程有 p1 和 p2，假设先让 p1 完成，它完成后系统资源剩余量为 2、3、5、7。最后资源满足 p2 的要求。由此可见 p2 的请求可以给予满足。

三、处理机管理

1. 什么是分级调度?分时系统中有作业调度的概念吗?如果没有,为什么?

答:处理机调度问题实际上也是处理机的分配问题。显然只有那些参与竞争处理及所必需的资源都已得到满足的进程才能享有竞争处理机的资格。这时它们处于内存就绪状态。这些必需的资源包括内存、外设及有关数据结构等。从而,在进程有资格竞争处理机之前,作业调度程序必须先调用存储管理、外设管理程序,并按一定的选择顺序和策略从输入井中选择出几个处于后备状态的作业,为它们分配资源和创建进程,使它们获得竞争处理机的资格。另外,由于处于执行状态下的作业一般包括多个进程,而在单机系统中,每一时刻只能有一个进程占有处理机,这样,在外存中,除了处于后备状态的作业外,还存在处于就绪状态而等待得到内存的作业。我们需要有一定的方法和策略为这部分作业分配空间。因此处理机调度需要分级。

一般来说,处理机调度可分为4级;

- (1)作业调度:又称宏观调度,或高级调度。
- (2)交换调度:又称中级调度。其主要任务是按照给定的原则和策略,将处于外存交换区中的就绪态或等待状态或内存等待状态的进程交换到外存交换区。交换调度主要涉及到内存管理与扩充。因此在有些书本中也把它归入内存管理部分。
- (3)进程调度:又称微观调度或低级调度。其主要任务是按照某种策略和方法选取一个处于就绪状态的进程占用处理机。在确立了占用处理机的进程之后,系统必须进行进程上下文切换以建立与占用处理机进程相适应的执行环境。
- (4)线程调度:进程中相关堆栈和控制表等的调度。

在分时系统中,一般不存在作业调度,而只有线程调度、进程调度和交换调度。这是因为在分时系统中,为了缩短响应时间,作业不是建立在外存,而是直接建立在内存中。在分时系统中,一旦用户和系统的交互开始,用户马上要进行控制。因此,分时系统中没有作业提交

状态和后备状态。分时系统的输入信息经过终端缓冲区为系统直接接收，或立即处理，或经交换调度暂存外存中。

2.

2. 将一组进程分为 4 类，如图 2-6 所示。各类进程之间采用优先级调度算法，而各类进程的
内部采用时间片轮转调度算法。请简述 P1、P2、P3、P4、P5、P6、P7、P8 进程的调度过程。

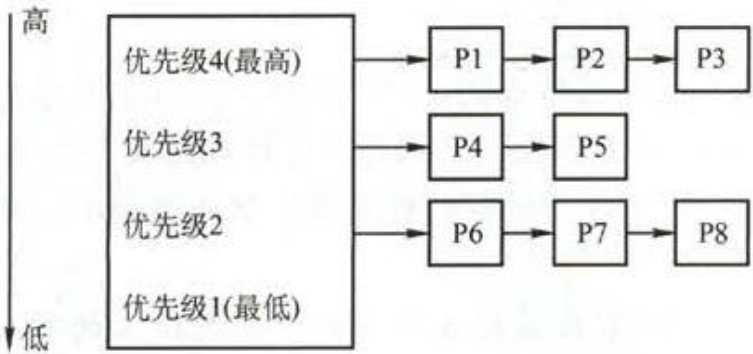


图 2-6 优先级调度示意图

答：

2. 解答：

从题意可知，各类进程之间采用优先级调度算法，而同类进程内部采用时间片轮转调度算法，因此，系统首先对优先级为 4 的进程 P1、P2、P3 采用时间片轮转调度算法运行；当 P1、P2、P3 均运行结束或没有可运行的进程（即 P1、P2、P3 都处于等待状态；或其中部分进程已运行结束，其余进程处于等待状态）时，则对优先级为 3 的进程 P4、P5 采用时间片轮转调度算法运行。在此期间，如果未结束的 P1、P2、P3 有一个转为就绪状态，则当前时间片用完后又回到优先级 4 进行调度。类似地，当 P1~P5 均运行结束或没有可运行进程（即 P1~P5 都处于等待状态；或其中部分进程已运行结束，其余进程处于等待状态）时，则对优先级为 2 的进程 P6、P7、P8 采用时间片轮转调度算法运行，一旦 P1~P5 中有一个转为就绪状态，则当前时间片用完后立即回到相应的优先级进行时间片轮转调度。

3.

有一个 CPU 和两台外设 D1、D2，且能够实现抢占式优先级调度算法的多道程序环境中，同时进入优先级由高到低的 P1、P2、P3 三个作业，每个作业的处理顺序和使用资源的时间如下：

P1: D2(30ms), CPU(10ms), D1(30ms), CPU(10ms)

P2: D1(20ms), CPU(20ms), D2(40ms)

P3: CPU(30ms), D1(20ms)

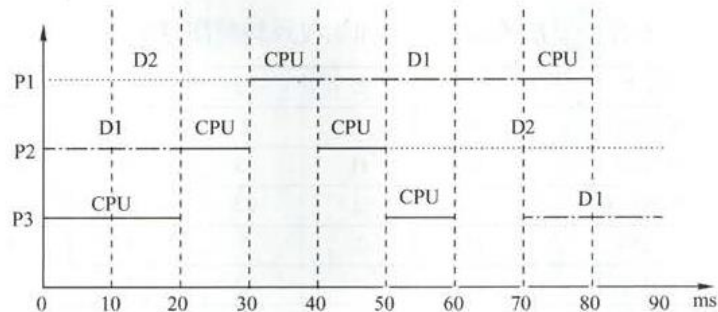
假设对于其他辅助操作时间忽略不计，每个作业的周转时间 T1、T2、T3 分别为多少？CPU 和 D1 的利用率各是多少？

答：

解答：

抢占式优先级调度算法，三个作业执行的顺序如下图所示。

作业 P1 的优先级最高，所以周转时间等于运行时间， $T_1=80\text{ms}$ ；作业 P2 等待时间为 10ms ，运行时间为 80ms ，故周转时间 $T_2=(10+80)\text{ms}=90\text{ms}$ ；作业 P3 的等待时间为 40ms ，运行时间为 50ms ，故周转时间 $T_3=90\text{ms}$ 。



三个作业从进入系统到全部运行结束，时间为 90ms 。CPU 与外设都是独占设备，运行时间分别为各作业的使用时间之和：CPU 运行时间为 $[(10+10)+20+30]\text{ms}=70\text{ms}$ ，D1 为 $(30+20+20)\text{ms}=70\text{ms}$ ，D2 为 $(30+40)\text{ms}=70\text{ms}$ 。故利用率均为 $70/90=77.8\%$ 。

4. 假定要在—台处理机上执行下表中所示的作业，且假定这些作业在时刻 0 以 1、2、3、4、5 的顺序到达，说明分别使用 FCFS、RR（时间片=1）、SJF 以及非剥夺优先级调度算法时，这些作业的执行情况。（优先级的高低顺序依次为 1 到 5），同时针对以上每种调度算法，给出平均周转时间和平均带权周转时间。

作业	执行时间	优先级
1	10	3
2	1	1
3	2	3
4	1	4
5	5	2

答：

(1) 作业执行情况可以用甘特图来表示。

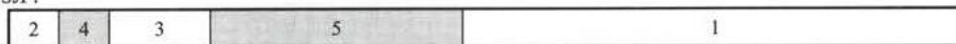
FCFS:



RR:



SJF:



优先级:



(2) 各个作业对应于各个算法的周转时间和加权周转时间见下表。

算法	时间类型	P1	P2	P3	P4	P5	平均时间
	运行时间	10	1	2	1	5	3.8
FCFS	周转时间	10	11	13	14	19	13.4
	加权周转时间	1	11	6.5	14	3.8	7.26
RR	周转时间	19	2	7	4	14	9.2
	加权周转时间	1.9	2	3.5	4	2.8	2.84
SJF	周转时间	19	1	4	2	9	7
	加权周转时间	1.9	1	2	2	1.8	1.74
优先级	周转时间	16	1	18	19	6	12
	加权周转时间	1.6	1	9	19	1.2	6.36

所以, FCFS 的平均周转时间为 13.4, 平均加权周转时间为 7.26。

RR 的平均周转时间为 9.2, 平均加权周转时间为 2.84。

SJF 的平均周转时间为 7, 平均加权周转时间为 1.74。

非剥夺式优先级调度算法的平均周转时间为 12, 平均加权周转时间为 6.36。

注意: SJF 的平均周转时间肯定是最短的, 计算完后可以利用这个性质检验。

5.

有一个具有两道作业的批处理系统, 作业调度采用短作业优先调度算法, 进程调度采用

抢占式优先级调度算法。作业的运行情况见表 2-7, 其中作业的优先数即为进程的优先数, 优先数越小, 优先级越高。问:

1) 列出所有作业进入内存的时间及结束的时间 (以分钟为单位);

2) 计算平均周转时间。

表 2-7 作业运行情况

作业名	到达时间	运行时间	优先数
1	8:00	40 分钟	5
2	8:20	30 分钟	3
3	8:30	50 分钟	4
4	8:50	20 分钟	6

答:

解答：

1) 具有两道作业的批处理系统，内存只存放两道作业，它们采用抢占式优先级调度算法竞争 CPU，而将作业调入内存则采用的是短作业优先调度。在 8:00 时，作业 1 到来，此时内存和处理机空闲，作业 1 进入内存并占用处理机；8:20 时，作业 2 到来，内存仍有一个位置空闲，故将作业 2 调入内存，又由于作业 2 的优先数高，相应的进程抢占处理机，在此期间 8:30 作业 3 到来，但内存此时已无空闲，故等待。直至 8:50，作业 2 执行完毕，此时作业 3、4 竞争空出的一道内存空间，作业 4 的运行时间短，故先调入，但它的优先数低于作业 1，故作业 1 先执行。到 9:10 时，作业 1 执行完毕，再将作业 3 调入内存，且由于作业 3 的优先数高而占用 CPU。所有作业进入内存的时间及结束的时间见下表。

作业	到达时间	运行时间	优先数	进入内存时间	结束时间	周转时间
1	8:00	40min	5	8:00	9:10	70min
2	8:20	30min	3	8:20	8:50	30min
3	8:30	50min	4	9:10	10:00	50min
4	8:50	20min	6	8:50	10:20	90min

2) 平均周转时间为：(70+30+50+90)/4=60(min)。

6. 何谓死锁?产生死锁的原因和必要条件是什么?

答：a. 死锁是指多个进程因竞争资源而造成的一种僵局，若无外力作用，这些进程都将永远不能再向前推进；b. 产生死锁的原因有二，一是竞争资源，二是进程推进顺序非法；c. 必要条件是：互斥条件，请求和保持条件，不剥夺条件和环路等待条件。

7. 请详细说明可通过哪些途径预防死锁。

答：a. 摒弃"请求和保持"条件，就是如果系统有足够的资源，便一次性地把进程所需的所有资源分配给它；b. 摒弃"不剥夺"条件，就是已经保持了资源的进程，当它提出新的资源请求而不能立即得到满足时，必须释放它已经保持的所有资源，待以后需要时再重新申请；c. 摒弃"环路等待"条件，就是将所有资源按类型排序标号，所有进程对资源的请求必须严格按序号递增次序提出。

8.

某系统有 R1、R2 和 R3 共三种资源，在 T₀ 时刻 P1、P2、P3 和 P4 这四个进程对资源的占用和需求情况见表 2-18，此时系统的可用资源矢量为(2, 1, 2)。试问：

1) 将系统中各种资源总数和此刻各进程对各资源的需求数目用矢量或矩阵表示出来。

2) 如果此时进程 P1 和进程 P2 均发出资源请求矢量 Request(1, 0, 1)，为了保证系统的安全性，应如何分配资源给这两个进程？说明所采用策略的原因。

表 2-18 T₀ 时刻四个进程对资源的占用和需求情况

资源情况 进程	最大资源需求量			已分配资源数量		
	R1	R2	R3	R1	R2	R3
P1	3	2	2	1	0	0
P2	6	1	3	4	1	1
P3	3	1	4	2	1	1
P4	4	2	2	0	0	2

3) 如果 2) 中两个请求立即得到满足后，系统此刻是否处于死锁状态？

答：

1) 系统中资源总量为某时刻系统中可用资源量与各进程已分配资源量之和, 即 $(2, 1, 2) + (1, 0, 0) + (4, 1, 1) + (2, 1, 1) + (0, 0, 2) = (9, 3, 6)$, 各进程对资源的需求量为各进程对资源的最大需求量与进程已分配资源量之差, 即

$$\begin{pmatrix} 3 & 2 & 2 \\ 6 & 1 & 3 \\ 3 & 1 & 4 \\ 4 & 2 & 2 \end{pmatrix} - \begin{pmatrix} 1 & 0 & 0 \\ 4 & 1 & 1 \\ 2 & 1 & 1 \\ 0 & 0 & 2 \end{pmatrix} = \begin{pmatrix} 2 & 2 & 2 \\ 2 & 0 & 2 \\ 1 & 0 & 3 \\ 4 & 2 & 0 \end{pmatrix}$$

2) 若此时 P1 发出资源请求 $\text{Request}_1(1, 0, 1)$, 按银行家算法进行检查:

$$\text{Request}_1(1, 0, 1) \leq \text{Need}_1(2, 2, 2)$$

$$\text{Request}_1(1, 0, 1) \leq \text{Available}(2, 1, 2)$$

试分配并修改相应数据结构, 由此形成的进程 P1 请求资源后的资源分配情况见下表。

资源情况 进程	Allocation			Need			Available		
P1	2	0	1	1	2	1	1	1	1
P2	4	1	1	2	0	2			
P3	2	1	1	1	0	3			
P4	0	0	2	4	2	0			

再利用安全性算法检查系统是否安全, 可用资源 $\text{Available}(1, 1, 1)$ 已不能满足任何进程, 系统进入不安全状态, 此时系统不能将资源分配给进程 P1。

若此时进程 P2 发出资源请求 $\text{Request}_2(1, 0, 1)$, 按银行家算法进行检查:

$$\text{Request}_2(1, 0, 1) \leq \text{Need}_2(2, 0, 2)$$

$$\text{Request}_2(1, 0, 1) \leq \text{Available}(2, 1, 2)$$

试分配并修改相应数据结构, 由此形成的进程 P2 请求资源后的资源分配情况见下表:

资源情况 进程	Allocation			Need			Available		
P1	1	0	0	2	2	2	1	1	1
P2	5	1	2	1	0	1			
P3	2	1	1	1	0	3			
P4	0	0	2	4	2	0			

再利用安全性算法检查系统是否安全, 可得到如下表中所示的安全性检测情况。注意表中各个进程对应的 Available 向量表示在该进程释放资源之后更新的 Available 向量。

资源情况 进程	Work			Need			Allocation			Available		
P2	1	1	1	1	0	1	5	1	2	6	2	3
P3	6	2	3	1	0	3	2	1	1	8	3	4
P4	8	3	4	4	2	0	0	0	2	8	3	6
P1	8	3	6	2	2	2	1	0	0	9	3	6

从上表中可以看出, 此时存在一个安全序列 $\{P2, P3, P4, P1\}$, 故该状态是安全的, 可以立即将进程 P2 所申请的资源分配给它。

如果 2) 中的两个请求立即得到满足, 此刻系统并没有立即进入死锁状态, 因为这时所有的进程没有提出新的资源申请, 全部进程均没有因资源请求没得到满足而进入阻塞状态。只有当进程提出资源申请且全部进程都进入阻塞状态时, 系统才处于死锁状态。

9. 何谓作业、作业步和作业流？

答：作业包含通常的程序和数据，还配有作业说明书。系统根据该说明书对程序的运行进行控制。批处理系统中是以作业为基本单位从外存调入内存。

作业步是指每个作业运行期间都必须经过若干个相对独立相互关联的顺序加工的步骤。

作业流是指若干个作业进入系统后依次存放在外存上形成的输入作业流；在操作系统的控制下，逐个作业进程处理，于是形成了处理作业流。

10.

考虑某个系统在表 2-19 时刻的状态。

表 2-19 系统资源状态表

	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2	1	5	2	0
P1	1	0	0	0	1	7	5	0				
P2	1	3	5	4	2	3	5	6				
P3	0	0	1	4	0	6	5	6				

使用银行家算法回答下面的问题：

1) Need 矩阵是怎样的？

2) 系统是否处于安全状态？如安全，请给出一个安全序列。

3) 如果从进程 P1 发来一个请求 (0, 4, 2, 0)，这个请求能否立刻被满足？如安全，请给出一个安全序列。

答：

(1)

$$\text{Need} = \text{Max} - \text{Allocation} = \begin{pmatrix} 0 & 0 & 1 & 2 \\ 1 & 7 & 5 & 0 \\ 2 & 3 & 5 & 6 \\ 0 & 6 & 5 & 6 \end{pmatrix} - \begin{pmatrix} 0 & 0 & 1 & 2 \\ 1 & 0 & 0 & 0 \\ 1 & 3 & 5 & 4 \\ 0 & 0 & 1 & 4 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 7 & 5 & 0 \\ 1 & 0 & 0 & 2 \\ 0 & 6 & 4 & 2 \end{pmatrix}$$

(2) Work 矢量初始化值=Available(1, 5, 2, 0)

系统安全性分析:

资源情况 进程	Work				Need				Allocation				Available			
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D
P0	1	5	2	0	0	0	0	0	0	0	1	2	1	5	3	2
P2	1	5	3	2	1	0	0	2	1	3	5	4	2	8	8	6
P1	2	8	8	6	0	7	5	0	1	0	0	0	3	8	8	6
P3	3	8	8	6	0	6	4	2	0	0	1	4	3	8	9	10

因为存在一个安全序列<P0、P2、P1、P3>，所以系统处于安全状态。

(3)

Request₁(0, 4, 2, 0)<Need₁(0, 7, 5, 0)

Request₁(0, 4, 2, 0)<Available(1, 5, 2, 0)

假设先试着满足进程 P1 的这个请求，则 Available 变为(1, 1, 0, 0)

系统状态变化见下表:

资源情况 进程	Max				Allocation				Need				Available			
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2	0	0	0	0	1	1	0	0
P1	1	7	5	0	1	4	2	0	0	3	3	0				
P2	2	3	5	6	1	3	5	4	1	0	0	2				
P3	0	6	5	6	0	0	1	4	0	6	4	2				

再对系统进行安全性分析，见下表:

资源情况 进程	Work				Need				Allocation				Available			
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D
P0	1	1	0	0	0	0	0	0	0	0	1	2	1	1	1	2
P2	1	1	1	2	1	0	0	2	1	3	5	4	2	4	6	6
P1	2	4	6	6	0	3	3	0	1	4	2	0	3	8	8	6
P3	3	8	8	6	0	6	4	2	0	0	1	4	3	8	9	10

因为存在一个安全序列<P0、P2、P1、P3>，所以系统仍处于安全状态。所以进程 P1 的这个请求应该马上被满足。

11. 试比较 FCFS 和 SPF 两种进程调度算法。

答：

相同点：两种调度算法都可以用于作业调度和进程调度。

不同点：FCFS 调度算法每次都从后备队列中选择一个或多个最先进入该队列的作业，将它们调入内存、分配资源、创建进程、插入到就绪队列。该算法有利于长作业/进程，不利于短作业/进程。SPF 算法每次调度都从后备队列中选择一个或若干个估计运行时间最短的作业，调入内存中运行。该算法有利于短作业/进程，不利于长作业/进程。

12. 几种调度算法（内含计算题）：

先来先服务（FCFS）调度算法的实现思想：按作业（进程）到来的先后次序进行调度，即先来的先得到运行。

用于作业调度：从作业队列（按时间先后为序）中选择队头的一个或几个作业运行。

用于进程调度：从就绪队列中选择一个最先进入该队列的进程投入运行。

计算题：设有三个作业，编号为 1，2，3。各作业分别对应一个进程。各作业依次到达，相差一个时间单位。算出各作业的周转时间和带权周转时间

作业	到达时间	运行时间	开始时间	完成时间	周转时间	带权周转时间
1	0	24	0	24	24	1
2	1	3	24	27	26	8.67
3	2	3	27	30	28	9.33
平均周转时间 $T=26$					平均带权周转时间 $W=6.33$	

（2）时间片轮转（RR）调度算法的实现思想：系统把所有就绪进程按先进先出的原则排成一个队列。新来的进程加到就绪队列末尾。每当执行进程调度时，进程调度程序总是选出就绪队列的队首进程，让它在 CPU 上运行一个时间片的时间。当时间片到，产生时钟中断，调度程序便停止该进程的运行，并把它放入就绪队列末尾，然后，把 CPU 分给就绪队列的队首进程。

时间片：是一个小的时间单位,通常 10~100ms 数量级。

计算题：设四个进程 A、B、C 和 D 依次进入就绪队列（同时到达），四个进程分别需要运行 12、5、3 和 6 个时间单位。

计算出各进程的周转时间和带权周转时间

进程名 到达时间	到达 时间	运行 时间	开始 时间	完成 时间	周转 时间	带权周转 时间
时间片 q=1	A	0	12	0	26	2.17
	B	0	5	1	17	3.4
	C	0	3	2	11	3.67
	D	0	6	3	20	3.33
	平均周转时间 T=18.5 平均带权周转时间 W=3.14					
时间片 q=4	A	0	12	0	26	2.17
	B	0	5	4	20	4
	C	0	3	8	11	3.67
	D	0	6	11	22	3.67
	平均周转时间 T=19.75 平均带权周转时间 W=3.38					

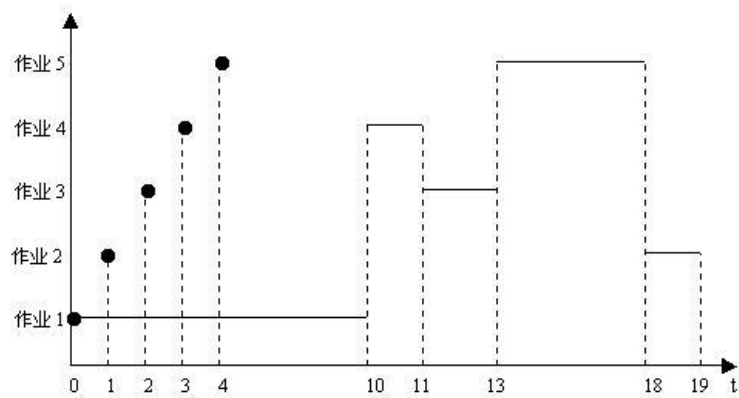
（3）优先级调度算法的实现思想：从就绪队列中选出优先级最高的进程到 CPU 上运行。

- 1) 两种不同的处理方式：非抢占式优先级法、抢占式优先级法
- 2) 两种确定优先级的方式：静态优先级、动态优先级

计算题：假定在单 CPU 条件下有下列要执行的作业：

作 业	运行 时 间	优先 级
1	10	3
2	1	1
3	2	3
4	1	4
5	5	2

① 用执行时间图描述非强占优先级调度算法执行这些作业的情况



算出各作业的周转时间和带权周转时间

作业	到达时间	运行时间	开始时间	完成时间	周转时间	带权周转时间
1	0	10	0	10	10	1 . 0
2	1	1	18	19	18	18 . 0
3	2	2	11	13	11	5 . 5
4	3	1	10	11	8	8 . 0
5	4	5	13	18	14	28
平均周转时间 T=12 . 2						平均带权周转时间 W=7 . 06

四、存储器管理

1. 某系统的空闲分区如表所示，采用可变式分区管理策略（动态分区管理策略），现有如下作业序列：96KB、20KB、200KB。若用首次适应算法和最佳适应算法来处理这些作业，则哪一种算法可满足该作业序列请求，为什么？【王道 2019，P163】

分区号	大小	起始地址
1	32KB	100KB
2	10KB	150KB
3	5KB	200KB
4	218KB	220KB
5	96KB	530KB

解：

采用首次适应算法时，96KB 大小的作业进入 4 号空闲分区，20KB 大小的作业进入 1 号空闲分区，这时空闲分区如下表所示。

分区号	大小	起始地址
1	12KB	120KB
2	10KB	150KB
3	5KB	200KB
4	122KB	316KB
5	96KB	530KB

此时再无空闲分区可以满足 200KB 大小的作业，所以该作业序列请求无法满足。

采用最佳适应算法时，作业序列分别进入 5、1、4 号空闲分区，可以满足其请求。分配处理之后的空闲分区表见下表：

分区号	大小	起始地址
1	12KB	120KB
2	10KB	150KB
3	5KB	200KB
4	18KB	420KB

2. 页式存储管理, 允许用户编程空间为 32 个页面 (每个页面 1KB), 主存为 16KB, 如有一用户程序有 10 页长, 且某一时刻该用户程序页表如下所示。如果分别遇到以下三个逻辑地址: 0AC5H、1AC5H、3AC5H 处的操作, 试计算其说明存储管理系统如何处理。【王道 2019, P164】

逻辑页号	物理块号
0	8
1	7
2	4
3	10

解:

页面大小为 1KB, 所以低 10 位为页内偏移地址; 用户编程空间为 32 个页面, 即逻辑地址高 5 位为虚页号; 主存为 16 个页面, 即物理地址高 4 位为物理块号。

逻辑地址 0AC5H 转换为二进制为 **000 1010 1100 0101B**, 虚页号为 2(00010B), 映射至物理块号 4, 故系统访问物理地址 12C5H(**01 0010 1100 0101B**)。

逻辑地址 1AC5H 转换为二进制为 **001 1010 1100 0101B**, 虚页号为 6(00110B), 不在页面映射表中, 会产生缺页中断, 系统进行缺页中断处理。

逻辑地址 3AC5H 转换为二进制为 **011 1010 1100 0101B**, 页号为 14, 而该用户程序只有 10 页, 故系统产生越界中断。

注意: 题中在对十六进制地址转换为二进制时, 我们可能会习惯性地写为 16 位, 这是容易犯错的细节。如题中逻辑地址是 15 位, 物理地址为 14 位。逻辑地址 0AC5H 的二进制表示为 000 1010 1100 0101B, 对应物理地址 12C5H 的二进制表示为 01 0010 1100 0101B。这一点应该注意。

3. 【王道 2019, P165】

11. 在页式、段式和段页式存储管理中, 当访问一条指令或数据时, 各需要访问内存几次? 其过程如何? 假设一个页式存储系统具有快表, 多数活动页表项都可以存在其中。如果页表存放在内存中, 内存访问时间是 $1\mu\text{s}$, 检索快表的时间为 $0.2\mu\text{s}$, 若快表的命中率是 85%, 则有效存取时间是多少? 若快表的命中率为 50%, 那么有效存取时间是多少?

解:

1) 页式存储管理中, 访问指令或数据时, 首先要访问内存中的页表, 查找到指令或数据所在页面对应的页表项, 然后再根据页表项查找访问指令或数据所在的内存页面。需要访问内存两次。

段式存储管理同理, 需要访问内存两次。

段页式存储管理, 首先要访问内存中的段表, 然后再访问内存中的页表, 最后访问指令或数据所在的内存页面。需要访问内存三次。

对于比较复杂的情况, 如多级页表, 若页表划分为 N 级, 则需要访问内存 $N+1$ 次。若系统中有快表, 则在快表命中时, 只需要一次访问内存即可。

2) 按 1) 中的访问过程分析, 有效存取时间为:

$$(0.2+1) \times 85\% + (0.2+1+1) \times (1-85\%) = 1.35(\mu s)$$

3) 同理可计算得:

$$(0.2+1) \times 50\% + (0.2+1+1) \times (1-50\%) = 1.7(\mu s)$$

从结果可以看出, 快表的命中率对访存时间影响非常大。当命中率从 85% 降低到 50% 时, 有效存取时间增加一倍。因此在页式存储系统中, 应尽可能地提高快表的命中率, 从而提高系统效率。

注意: 在有快表的分页存储系统中, 计算有效存取时间时, 需注意访问快表与访问内存的时间关系。通常的系统中, 先访问快表, 未命中时再访问内存; 在有些系统中, 快表与内存的访问同时进行, 当快表命中时就停止对内存的访问。这里题中未具体指明, 我们按照前者进行计算。但如果题中有具体的说明, 计算时则应注意区别。

4. 【王道 2019, P192】

6. 在一个请求分页存储管理系统中, 一个作业的页面走向为 4, 3, 2, 1, 4, 3, 5, 4, 3, 2, 1, 5, 当分配给作业的物理块数分别为 3 和 4 时, 试计算采用下述页面淘汰算法时的缺页率 (假设开始执行时主存中没有页面), 并比较结果。

1) 最佳置换算法;

2) 先进先出置换算法;

3) 最近最久未使用算法。

解:

1) 根据页面走向, 使用最佳置换算法时, 页面置换情况见下表。

物理块数为 3 时:

走向	4	3	2	1	4	3	5	4	3	2	1	5
块 1	4	4	4	4	4	4	4	4	4	2	2	2
块 2		3	3	3	3	3	3	3	3	3	1	1
块 3			2	1	1	1	5	5	5	5	5	5
缺页	√	√	√	√			√			√	√	

缺页率为: 7/12。

物理块数为 4 时:

走向	4	3	2	1	4	3	5	4	3	2	1	5
块 1	4	4	4	4	4	4	4	4	4	4	1	1
块 2		3	3	3	3	3	3	3	3	3	3	3
块 3			2	2	2	2	2	2	2	2	2	2
块 4				1	1	1	5	5	5	5	5	5
缺页	√	√	√	√			√				√	

缺页率为: 6/12。

由上述结果可以看出, 增加分配作业的内存块数可以降低缺页率。

2) 根据页面走向, 使用先进先出页面淘汰算法时, 页面置换情况见下表。

物理块数为 3 时:

走向	4	3	2	1	4	3	5	4	3	2	1	5
块 1	4	4	4	1	1	1	5	5	5	5	5	
块 2		3	3	3	4	4	4	4	4	2	2	
块 3			2	2	2	3	3	3	3	3	1	
缺页	√	√	√	√	√	√	√			√	√	

缺页率为: 9/12。

物理块数为 4 时:

走向	4	3	2	1	4	3	5	4	3	2	1	5
块 1	4	4	4	4	4	4	5	5	5	5	1	1
块 2		3	3	3	3	3	3	4	4	4	4	5
块 3			2	2	2	2	2	2	3	3	3	3
块 4				1	1	1	1	1	1	2	2	2
缺页	√	√	√	√			√	√	√	√	√	√

缺页率为：10/12。

由上述结果可以看出，对先进先出算法而言，增加分配作业的内存块数反而使缺页率上升，即出现 Belady 现象。

3) 根据页面走向，使用最近最久未使用页面淘汰算法时，页面置换情况见下表。

物理块数为 3 时：

走向	4	3	2	1	4	3	5	4	3	2	1	5
块 1	4	4	4	1	1	1	5	5	5	2	2	2
块 2		3	3	3	4	4	4	4	4	4	1	1
块 3			2	2	2	3	3	3	3	3	3	5
缺页	√	√	√	√	√	√	√			√	√	√

缺页率为：10/12。

物理块数为 4 时：

走向	4	3	2	1	4	3	5	4	3	2	1	5
块 1	4	4	4	4	4	4	4	4	4	4	4	5
块 2		3	3	3	3	3	3	3	3	3	3	3
块 3			2	2	2	2	5	5	5	5	1	1
块 4				1	1	1	1	1	1	2	2	2
缺页	√	√	√	√			√			√	√	√

缺页率为：8/12。

由上述结果可以看出，增加分配作业的内存块数可以降低缺页率。

5. 【王道 2019, P195】

某计算机系统按字节编址，采用二级页表的分页存储管理方式，虚拟地址格式如下所示：

10 位	10 位	12 位
页目录号	页表索引	页内偏移量

请回答下列问题

- (1) 页和页框的大小各为多少字节？进程的虚拟地址空间大小为多少页？
- (2) 假定页目录项和页表项均占 4 个字节，则进程的页目录和页表共占多少页？要求写出计算过程。
- (3) 若某指令周期内访问的虚拟地址为 0100 0000H 和 0111 2048H，则进行地址转换时共访问多少个二级表页？要求说明理由。

解：

- 1) 页和页框大小均为 4KB。进程的虚拟地址空间大小为 $2^{32}/2^{12}=2^{20}$ 页。
- 2) $(2^{10}*4)/2^{12}$ (页目录所占页数) + $(2^{20}*4)/2^{12}$ (页表所占页数) = 1025 页。
- 3) 需要访问一个二级页表。因为虚拟地址 0100 0000H 和 0111 2048H 的最高 10 位的值都是 4, 访问的是同一个二级页表。

6. 【王道 2019, P194】

设某计算机的逻辑地址空间和物理地址空间均为 64KB, 按字节编址。若某进程最多需要 6 页 (Page) 数据存储空间, 页的大小为 1KB, 操作系统采用固定分配局部置换策略为此进程分配 4 个页框 (Page Frame), 见表 3-19。在时刻 260 前的该进程访问情况见表 3-4 (访问位即使用位)。

表 3-19 为进程分配页框 (注: 页框即内存物理块)

页号	页框号	装入时刻	访问位
0	7	130	1
1	4	230	1
2	2	200	1
3	9	160	1

当该进程执行到时刻 260 时, 要访问逻辑地址为 17CAH 的数据。请回答下列问题:

- 1) 该逻辑地址对应的页号是多少?
- 2) 若采用先进先出 (FIFO) 置换算法, 该逻辑地址对应的物理地址是多少? 要求给出计算过程。若采用时钟 (Clock) 置换算法, 该逻辑地址对应的物理地址是多少? 要求给出计算过程。设搜索下一页的指针沿顺时针方向移动, 且当前指向 2 号页框。

解:

- 1) 由于该计算机的逻辑地址空间和物理地址空间均为 64KB = 2^{16} B, 按字节编址, 且页的大小为 1K = 2^{10} , 故逻辑地址和物理地址的地址格式均为:

页号/页框号 (6 位)	页内偏移量 (10 位)
--------------	--------------

17CAH = 0001 0111 1100 1010B, 可知该逻辑地址的页号为 000101B = 5。

- 2) 采用 FIFO 置换算法, 与最早调入的页面即 0 号页面置换, 其所在的页框号为 7, 于是对应的物理地址为:

0001 1111 1100 1010B = 1FCAH

- 3) 采用 CLOCK 置换算法, 首先从当前位置 (2 号页框) 开始顺时针寻找访问位为 0 的页面, 当指针指向的页面的访问位为 1 时, 就把该访问位清 “0”, 指针遍历一周后, 回到 2 号页框, 此时 2 号页框的访问位为 0, 置换该页框的页面, 于是对应的物理地址为:

0000 1011 1100 1010B = 0BCAH

7. 什么是段式管理? 其与页式管理的有何区别 (请至少举例说明三点)? (段式管理即分段存储管理, 页式管理即分页存储管理)

解: 段式管理就是将程序按照内容或过程 (函数) 关系分成段, 每段拥有自己的名字。一个用户作业或进程所包含的段对应于一个二维线性

虚拟空间，也就是一个二维虚拟存储器。段式管理程序以段为单位分配内存，然后通过地址映射机构把段式虚拟地址转换成实际的内存物理地址。同页式管理类似，段式管理也采用只把那些经常访问的段驻留内存，而把那些在将来一段时间内不被访问的段放入外存，待需要时自动调入相关段的方法实现二维虚拟存储器。

段式管理和页式管理的主要区别有：

(1) 页式管理中源程序进行编译链接时是将主程序、子程序、数据区等按照线性空间的一维地址顺序排列起来。段式管理则是将程序按照内容或过程(函数)关系分成段，每段拥有自己的名字。一个用户作业或进程所包含的段对应于一个二维线性虚拟空间，也就是一个二维虚拟存储器。

(2) 当实现虚拟存储功能时，段式管理与页式管理有所不同。段式虚存每次交换的是一段有意义的信息；而不是像页式虚存管理那样只交换固定大小的页，需要多次的缺页中断才能把所需信息完整地调入内存。

(3) 在段式管理中，段长可根据需要动态增长。这对那些需要不断增加或改变新数据或子程序的段来说，将是非常有好处的。

(4) 段式管理便于对具有完整逻辑功能的信息段进行共享。

(5) 段式管理便于进行动态链接，而页式管理进行动态链接的过程非常复杂。

8. 试全面比较连续分配和离散分配存储器管理方式。

解：

连续分配是指为一个用户程序分配一个连续的地址空间，包括单一连续分配方式和分区式分配方式。

(1) 单一连续分配方式将内存分为系统区和用户区，系统区供操作系统使用，用户区供用户使用，是最简单的一种存储方式，但只能用于单用户单任务的操作系统中

(2) 分区式分配方式分为固定分区和动态分区。

a. 固定分区是最简单的多道程序的存储管理方式，由于每个分区的大小固定，必然会造成存储空间的浪费；

b. 动态分区是根据进程的实际需要, 动态地为之分配连续的内存空间, 常用三种分配算法: 首次适应算法, 该法容易留下许多难以利用的小空闲分区, 加大查找开销; 循环首次适应算法, 该算法能使内存中的空闲分区分布均匀, 但会致使缺少大的空闲分区; 最佳适应算法, 该算法也易留下许多难以利用的小空闲区。

离散分配方式基于将一个进程直接分散地分配到许多不相邻的分区中的思想, 分为分页式存储管理、分段存储管理和段页式存储管理。

- (1) 分页式存储管理旨在提高内存利用率, 满足系统管理的需要
- (2) 分段式存储管理则旨在满足用户(程序员)的需要, 在实现共享和保护方面优于分页式存储管理
- (3) 而段页式存储管理则是将两者结合起来, 取长补短, 即具有分段系统便于实现, 可共享, 易于保护, 可动态链接等优点, 又能像分页系统那样很好的解决外部碎片的问题, 以及为各个分段可离散分配内存等问题, 显然是一种比较有效的存储管理方式;

9. 实现地址重定位的方法有哪几类? 试列举其各自的特点

解: 实现地址重定位的方法有两种: 静态地址重定位和动态地址重定位。

(1) 静态地址重定位是在程序执行之前由装配程序完成地址映射工作。静态重定位的优点是不需要硬件支持, 但是用静态地址重定位方法进行地址变换无法实现虚拟存储器。静态重定位的另一个缺点是必须占用连续的内存空间和难以做到程序和数据的共享。

(2) 动态地址重定位是在程序执行过程中, 在 CPU 访问内存之前由硬件地址变换机构将要访问的程序或数据地址转换成内存地址。动态地址重定位的主要优点有:

- ①可以对内存进行非连续分配。
- ②动态重定位提供了实现虚拟存储器的基础。
- ③动态重定位有利于程序段的共享。

10. 【王道 2019, P194】

14. 某系统有 4 个页框, 某个进程页面使用情况见表 3-18, 请问采用 FIFO、LRU、简单 CLOCK 和改进型 CLOCK 置换算法, 将会替换哪一页?

表 3-18 进程页面使用情况

页号	装入时间	上次引用时间	R	M
0	126	279	0	0
1	230	260	1	0
2	120	272	1	1
3	160	280	1	1

其中, R 是读标志位, M 是修改标志位。

解:

1) FIFO 置换算法选择最先进入内存的页面进行替换。由表中装入时间可知, 第 2 页最先进入内存, 故 FIFO 置换算法将选择第 2 页替换。

2) LRU 置换算法选择最近最长时间未使用的页面进行替换。由表中上次引用时间可知, 第 1 页是最长时间未使用的页面, 故 LRU 置换算法将选择第 1 页替换。

3) 简单 CLOCK 置换算法从上一次位置开始扫描, 选择第一个访问位为 0 的页面进行替换。由表中 R (读) 标志位可知, 依次扫描 1、2、3、0 (按装入顺序), 页面 0 未被访问, 扫描结束, 故简单 CLOCK 置换算法将选择第 0 页替换。

4) 改进型 CLOCK 置换算法从上一次位置开始扫描, 首先寻找未被访问和修改的页面。由表中 R (读) 标志位和 M (修改) 标志位可知, 只有页面 0 满足 $R=0$ 和 $M=0$, 故改进型 CLOCK 置换算法将选择第 0 页替换。

11. 可采用哪几种方式将程序装入内存? 它们分别适用于何种场合? 课本 P132-133

答:

(1) 绝对装入方式, 只适用于单道程序环境。

(2) 可重定位装入方式, 适用于多道程序环境; 不允许程序运行时在内存中移位置。

(3) 动态运行时装入方式, 用于多道程序环境; 允许程序运行时在内存中移位置。

12. 何谓静态链接? 何谓装入时动态链接和运行时的动态链接?

答:

静态链接是指在程序运行前, 先将各目标模块及它们所需的库函数, 链接成一个完整的装配模块, 以后不再拆开的链接方式。

装入时动态链接是指将用户源程序编译后得到的一组目标模块, 在装入内存时采用边装入边链接的链接方式。

运行时动态链接是指对某些目标模块的链接, 是在程序执行中需要该

目标模块时，才对它进行的链接。

13. 分区存储管理中常用那些分配策略？比较它们的优缺点。

答：

分区存储管理中的常用分配策略：首次适应算法、循环首次适应算法、最佳适应算法、最坏适应算法。

首次适应算法优缺点：保留了高址部分的大空闲区，有利于后来的大型作业分配；低址部分不断被划分，留下许多难以利用的小空闲区，每次查找都从低址开始增加了系统开销。

循环首次适应算法优缺点：内存空闲分区分布均匀，减少了查找系统开销；缺乏大空闲分区，导致不能装入大型作业。

最佳适应算法优缺点：每次分配给文件的都是最适合该文件大小分区，内存中留下许多难以利用的小空闲区。

最坏适应算法优缺点：剩下空闲区不太小，产生碎片几率小，对中小型文件分配分区操作有利；存储器中缺乏大空闲区，对大型文件分区分配不利。

14. 分段和分页存储管理有何区别？

答：

(1) 页是信息的物理单位，分页是为了实现离散分配方式，以消减内存的外部零头，提高内存利用率。段则是信息的逻辑单位，它含有一组相对完整的信息。

(2) 页的大小固定且由系统决定，由系统把逻辑地址划分为页号和页内地址两部分，是由机械硬件实现的，因而在系统中只能有一种大小的页面；而段的长度却不固定，决定于用户所编写的程序，通常由编译程序在对原程序进行编译时，根据信息的性质来划分。

(3) 分页的作业地址空间是一维的，而分段作业地址空间则是二维的。

15. 在一个请求分页系统中，假设一个作业的页面走向为 4, 3, 2, 1, 4, 3, 5, 4, 3, 2, 1, 5，目前还没有任何页装入内存，当分配给该作业的物理块数目 M 分别为 3 和 4 时，请分别计算采用 OPT、LRU、FIFO 页面淘汰算法时访问过程中所发生的缺页次数和缺页率，并比较所得的结果。

如果访问的页还没装入主存，便将发生一次中断，访问过程中发生缺页中断的次数就是缺页次数，而缺页的次数除以总的访问次数，就是缺页率。

访问过程中的缺页情况 M=3 OPT

页面走向	4	3	2	1	4	3	5	4	3	2	1	5
缺页												
很长时间不用			2	1	1	1	5	4	4/3	3/2	1/2	
		3	3	3	4	3	3	5				
将来马上访问	4	4	4	4	3	4	4	3	5	5	5	

访问过程中的缺页情况 M=4 OPT

页面走向	4	3	2	1	4	3	5	4	3	2	1	5
缺页												
很长时间不用				1	1	1	5	4	4/3	4/3/2	1/3/2	
			2	2	2	2	2	5				
将来马上访问		3	3	3	4	3	3	2	5			
	4	4	4	4	3	4	4	3	2	5	5	

访问过程中的缺页情况 M=3 LRU

页面走向	4	3	2	1	4	3	5	4	3	2	1	5
缺页												
最近最长时间未用的内存页			4	3	2	1	4	3	5	4	3	2
		4	3	2	1	4	3	5	4	3	2	1
最近刚使用过的内存页	4	3	2	1	4	3	5	4	3	2	1	5

访问过程中的缺页情况 M=4 LRU

页面走向	4	3	2	1	4	3	5	4	3	2	1	5
缺页												
最近最长时间未用的内存页				4	3	2	1	1	1	5	4	3
			4	3	2	1	4	3	5	4	3	2
		4	3	2	1	4	3	5	4	3	2	1
最近刚使用过的内存页	4	3	2	1	4	3	5	4	3	2	1	5

访问过程中的缺页情况 M=3 FIFO

页面走向	4	3	2	1	4	3	5	4	3	2	1	5
缺页												
最早进入内存的页面			4	3	2	1	4	4	4	3	5	5
		4	3	2	1	4	3	3	3	5	2	2
	4	3	2	1	4	3	5	5	5	2	1	1
最晚进入内存的页面												

访问过程中的缺页情况 M=4 FIFO

页面走向	4	3	2	1	4	3	5	4	3	2	1	5
缺页												
最早进入内存的页面				4	3	2	1	1	1	5	4	3
			4	3	2	1	4	3	5	4	3	2
		4	3	2	1	4	3	5	4	3	2	1
最晚进入内存的页面	4	3	2	1	4	3	5	4	3	2	1	5

16. 有一请求分页存储管理系统，页面大小为每页 100 字节。有一个 50*50 的整型数组按行连续存放，每个整数占两个字节，将数组初始化为 0 的程序描述如下：

```

INT A[50][50]
INT I,J
for(I=0; i<=49; i++)
for (J=0; j<=49; j++)
a[i][J]=0;
```

若在程序执行时内存中只有一个存储块用来存放数组信息，试问该程序执行时产生多少次缺页中断？（连续存放）

解：块和页的大小相等，只要求出页数即可

由题目可知，该数组中有 2500 个整数，每个整数占用 2 个字节，

共需存储空间 5000 个字节：而页面大小为每页 100 字节，数组占用空间 50 页。假设数据从该作业的第 m 页开始存放，则数组分布在第 m 页到第 $m+49$ 页中，它在主存中的排列顺序为：

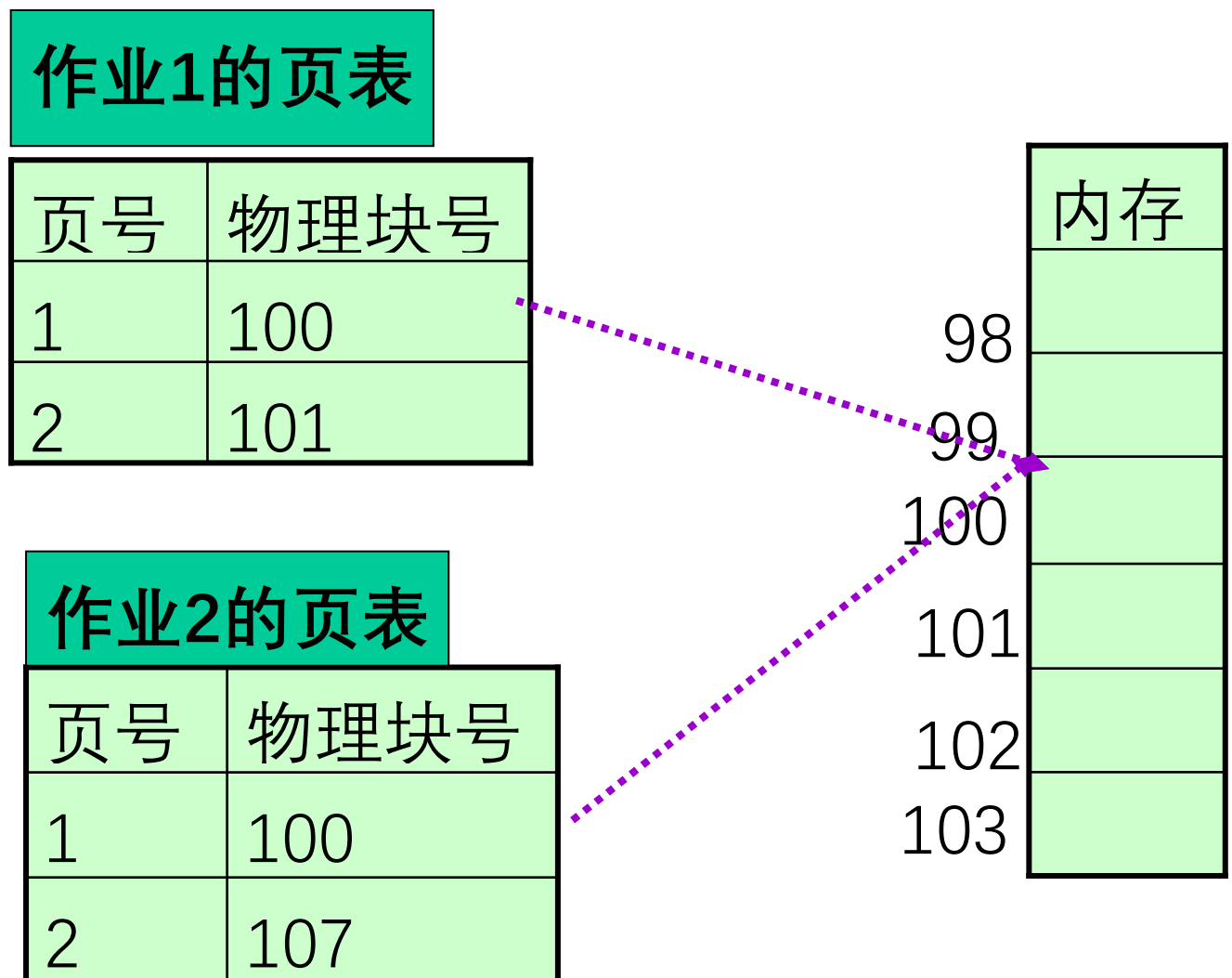
$A[0][0], A[0][1] \dots, a[0][49]$ 第 m 页

$A[1][0], A[1][1] \dots, a[1][49]$ 第 $m+1$ 页

$a[49][0], a[49][1], \dots, a[49][49]$ 第 $m+49$ 页

由于该初始化程序是按行进行的，因此每次缺页中断调进一页后，位于该页内的数组全部赋予 0 值，然后再调入下一页，所以涉及的页面走向为 $m, m+1, \dots, m+49$ ，故缺页次数为 50 次。

17、举例说明在分页系统中如何实现内存共享？要求图示说明



18、存储器有关概念

(1) 逻辑地址：用户程序经编译之后的每个目标模块都以 0 为基地址顺序编址。

(2) 物理地址：内存中各物理单元的地址是从统一的基地址顺序编址。

(3) 重定位：把逻辑地址转变为内存的物理地址的过程。

(4) 静态重定位：是在目标程序装入内存时，由装入程序对目标程序中的指令和数据的地址进行修改，即把程序的逻辑地址都改成实际的内存地址。重定位在程序装入时一次完成

(5) 动态重定位：在程序执行期间，每次访问内存之间进行重定位，这种变换是靠硬件地址变换机构实现的。

19、虚存中的置换算法

(1) 先进先出法 (FIFO)：将最先进入内存的页换出内存。

例如 内存块数量为 3 时，采用 FIFO 页面置换算法，下面页面走向情况下，缺页次数是多少？

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	2		2	2	4	4	4	0			0	0			7	7	7
	0	0	0		3	3	3	2	2	2			1	1			1	0	0
		1	1		1	0	0	0	3	3			3	2			2	2	1

∴ 缺页次数=15 次

(2) 最佳置换法 (OPT)：将将来不再被使用或是最远的将来才被访问的页

例如 内存块数量为 3 时，采用 OPT 页面置换算法，下面页面走向情

况下，缺页次数是多少？

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	2		2		2		2		2					7			
	0	0	0		0		4		0		0					0			
		1	1		3		3		3		1					1			

∴ 缺页次数=9 次

(3) 最近最少使用置换法 (LRU)：将最近一段时间里最久没有使用过的页面换出内存。

例如 内存块数量为 3 时，采用 LRU 页面置换算法，下面页面走向情况下，缺页次数是多少？

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	2		2		4	4	4	0			1		1		1		
	0	0	0		0		0	0	3	3			3		0		0		
		1	1		3		3	2	2	2			2		2		7		

∴ 缺页次数=12 次

(4) 最近未使用置换法 (NUR)：是 LRU 近似方法，比较容易实现，开销也比较小。 实现方法：在存储分块表的每一表项中增加一个引用位，操作系统定期地将它们置为 0。当某一页被访问时，由硬件将该位置 1。需要淘汰一页时，把该位为 0 的页淘汰出去，因为最近一段时间里它未被访问过。

五、设备管理

1. 数据传送控制方式有哪几种?试比较它们各自的优缺点。

答：数据传送控制方式有程序直接控制方式、中断控制方式、DMA 方式和通道方式 4 种。

(1) 程序直接控制方式就是由用户进程来直接控制内存或 CPU 和外围设备之间的数据传送。它的优点是控制简单,也不需要多少硬件支持。它的缺点是 CPU 浪费大量时间等待设备输入输出, CPU 和外围设备只能串行工作。每次传输的数据量以字节为单位,效率较低。

(2) 中断控制方式是利用向 CPU 发送中断的方式控制外围设备和 CPU 之间的数据传送。它的优点是大大提高了 CPU 的利用率且能支持多道程序和设备的并行操作。它的缺点是由于数据缓冲寄存器比较小,如果中断次数较多,仍然占用了大量 CPU 时间;在外围设备较多时,由于中断次数的急剧增加,可能造成 CPU 无法响应中断而出现中断丢失的现象;如果外围设备速度比较快,可能会出现 CPU 来不及从数据缓冲寄存器中取走数据而丢失数据的情况。每次传输的数据量以字节为单位,效率较低。

(3) DMA 方式是在外围设备和内存之间开辟直接的数据交换通路进行数据传送。它的优点是除了在数据块传送开始时需要 CPU 的启动指令,在整个数据块传送结束时需要发中断通知 CPU 进行中断处理之外,不需要 CPU 的频繁干涉;以数据块作为传送单位,效率较高。它的缺点是在外围设备越来越多的情况下,多个 DMA 控制器的同时使用,会引起内存地址的冲突并使得控制过程进一步复杂化。

(4) 通道方式是使用通道来控制内存或 CPU 和外围设备之间的数据传送。通道是一个独立与 CPU 的专门负责输入输出控制的机构,它控制设备与内存直接进行数据交换。它有自己的通道指令,这些指令受 CPU 启动,并在操作结束时向 CPU 发中断信号。该方式的优点是进一步减轻了 CPU 的工作负担,增加了计算机系统的并行工作程度;每次可以

传输一组数据块，效率最高。缺点是增加了额外的硬件，造价昂贵。

2. 试列举常用的磁盘调度算法并说明其主要特点

题目说明：图片仅供理解算法，考试时无需画图作答。

解：(1) 先来先服务 (First Come First Served, FCFS) 算法

FCFS 算法根据进程请求访问磁盘的先后顺序进行调度，这是一种最简单的调度算法，如图 4-25 所示。该算法的优点是具有公平性。如果只有少量进程需要访问，且大部分请求都是访问簇聚的文件扇区，则有望达到较好的性能；如果有大量进程竞争使用磁盘，那么这种算法在性能上往往接近于随机调度。所以，实际磁盘调度中考虑一些更为复杂的调度算法。

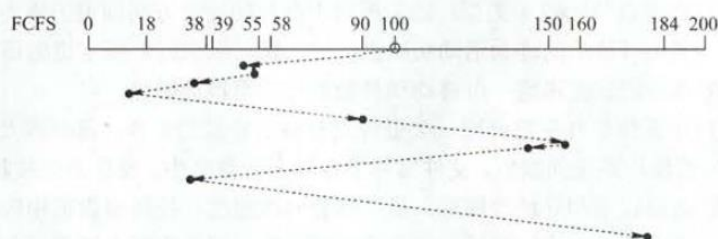


图 4-25 FCFS 磁盘调度算法

例如，磁盘请求队列中的请求顺序分别为 55、58、39、18、90、160、150、38、184，磁头初始位置是 100 磁道，采用 FCFS 算法磁头的运动过程如图 4-25 所示。磁头共移动了 $(45+3+19+21+72+70+10+112+146)=498$ 个磁道，平均寻找长度 $=498/9=55.3$ 。

(2) 最短寻找时间优先 (Shortest Seek Time First, SSTF) 算法

SSTF 算法选择调度处理的磁道是与当前磁头所在磁道距离最近的磁道，以使每次的寻找时间最短。当然，总是选择最小寻找时间并不能保证平均寻找时间最小，但是能提供比 FCFS 算法更好的性能。这种算法会产生“饥饿”现象。如图 4-26 所示，若某时刻磁头正在 18 号磁道，而在 18 号磁道附近频繁地增加新的请求，那么 SSTF 算法使得磁头长时间在 18 号磁道附近工作，将使 184 号磁道的访问被无限期地延迟，即被“饿死”。

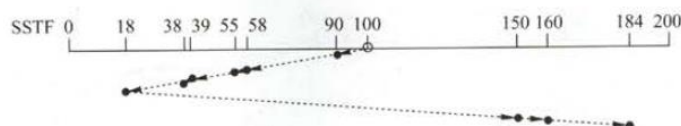


图 4-26 SSTF 磁盘调度算法

例如，磁盘请求队列中的请求顺序分别为 55、58、39、18、90、160、150、38、184，磁头初始位置是 100 磁道，采用 SSTF 算法磁头的运动过程如图 4-26 所示。磁头共移动了 $(10+32+3+16+1+20+132+10+24)=248$ 个磁道，平均寻找长度 $=248/9=27.5$ 。

(3) 扫描 (SCAN) 算法 (又称电梯算法)

SCAN 算法在磁头当前移动方向上选择与当前磁头所在磁道距离最近的请求作为下一次服务的对象,实际上就是在最短寻找时间优先算法的基础上规定了磁头运动的方向,如图 4-27 所示。由于磁头移动规律与电梯运行相似,故又称为电梯调度算法。SCAN 算法对最近扫描过的区域不公平,因此,它在访问局部性方面不如 FCFS 算法和 SSTF 算法好。

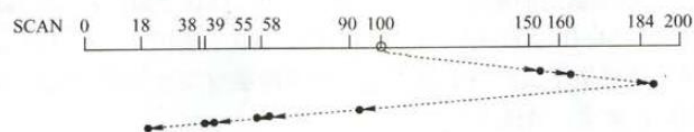


图 4-27 SCAN 磁盘调度算法

例如,磁盘请求队列中的请求顺序分别为 55、58、39、18、90、160、150、38、184,磁头初始位置是 100 磁道。采用 SCAN 算法时,不但要知道磁头的当前位置,还要知道磁头的移动方向,假设磁头沿磁道号增大的顺序移动,则磁头的运动过程如图 4-27 所示。移动磁道的顺序为 100、150、160、184、200、90、58、55、39、38、18。磁头共移动了 $(50+10+24+16+110+32+3+16+1+20)=282$ 个磁道,平均寻道长度 $=250/9=31.33$ 。

(4) 循环扫描 (Circular SCAN, C-SCAN) 算法

在扫描算法的基础上规定磁头单向移动来提供服务,回返时直接快速移动至起始端而不服务任何请求。由于 SCAN 算法偏向于处理那些接近最里或最外的磁道的访问请求,所以使用改进型的 C-SCAN 算法来避免这个问题。

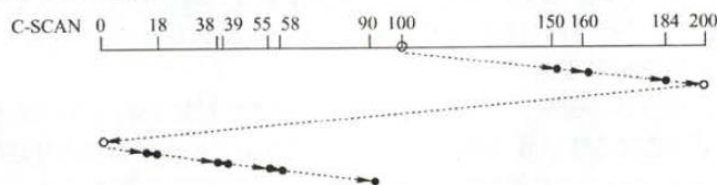


图 4-28 C-SCAN 磁盘调度算法

采用 SCAN 算法和 C-SCAN 算法时磁头总是严格地遵循从盘面的一端到另一端,显然,在实际使用时还可以改进,即磁头移动只需要到达最远端的一个请求即可返回,不需要到达磁盘端点。这种形式的 SCAN 算法和 C-SCAN 算法称为 LOOK (如图 4-27-1 所示) 和 C-LOOK (如图 4-28-1 所示) 调度。这是因为它们在朝一个给定方向移动前会查看是否有请求。

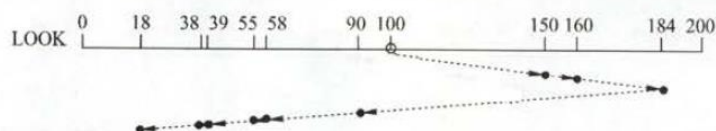


图 4-27-1 LOOK 磁盘调度算法

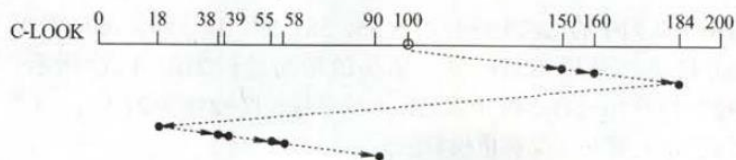


图 4-28-1 C-LOOK 磁盘调度算法

例如，磁盘请求队列中的请求顺序分别为 55、58、39、18、90、160、150、38、184，磁头初始位置是 100 磁道。采用 C-SCAN 算法时，假设磁头沿磁道号增大的顺序移动，则磁头的运动过程如图 4-28 所示。移动磁道的顺序为 100、150、160、184、200、0、18、38、39、55、58、90。磁头共移动 $50+10+24+16+200+18+20+1+16+3+32=390$ 个磁道，平均寻道长度 $=390/9=43.33$ 。

对于操作系统整体框架不太熟悉的读者经常会混淆磁盘调度算法中的循环扫描算法和页面调度算法中的 C-LOCK 算法，请读者注意区分。

3. 【王道 2019，P256】

在一个磁盘上，有 1000 个柱面，编号从 0~999，用下面的算法计算为满足磁盘队列中的所有请求，磁盘臂必须移过的磁道的数目。假设最后服务的请求是在磁道 345 上，并且读写头正在朝磁道 0 移动。在按 FIFO 顺序排列的队列中包含了如下磁道上的请求：

123、874、692、475、105、376

- (1) 先来先服务 (First Come First Served, FCFS) 算法
- (2) 最短寻找时间优先 (Shortest Seek Time First, SSTF) 算法
- (3) 扫描 (SCAN) 算法 (又称电梯算法)
- (4) LOOK (改进的 SCAN 算法，磁头移动只需要到达最远端的一个请求即可，不需要到达磁盘端点)
- (5) 循环扫描 (Circular SCAN, C-SCAN) 算法
- (6) C-LOOK (改进的 C-SCAN 算法，磁头移动只需要到达最远端的一个请求即可，不需要到达磁盘端点)

解：

1) **FCFS**: 移动磁道的顺序为 345、123、874、692、475、105、376。磁盘臂必须移过的磁道的数目为 $222+751+182+217+370+271=2013$ 。

2) **SSTF**: 移动磁道的顺序为 345、376、475、692、874、123、105。磁盘臂必须移过的磁道的数目为 $31+99+217+182+751+18=1298$ 。

注意: 磁盘臂必须移过的磁道的数目之和的计算没必要像上面一样对 31、99、217、182、751、18 求和, 仔细的读者会发现: 从 345 到 874 是一路递增的, 接着从 874 到 105 是一路递减的。所以仅需计算: $(874-345) + (874-105) = 1298$ 。是不是比上面的 6 个数的得出再计算它们的和要快捷一些? ! 若之前未注意到此法, 相信聪明的你会马上回顾刚作完的 1) 并会仔细观察以下几问的“规律”, 进而总结出自己的一番思路。

3) **SCAN**: 移动磁道的顺序为 345、123、105、0、376、475、692、874。磁盘臂必须移过的磁道的数目为 $222+18+105+376+99+217+182=1219$ 。

4) **LOOK**: 移动磁道的顺序为 345、123、105、376、475、692、874。磁盘臂必须移过的磁道的数目为 $222+18+271+99+217+182=1009$ 。

5) **C-SCAN**: 移动磁道的顺序为 345、123、105、0、999、874、692、475、376。磁盘臂必须移过的磁道的数目为 $222+18+105+999+125+182+217+99=1967$ 。

6) **C-LOOK**: 移动磁道的顺序为 345、123、105、874、692、475、376。磁盘臂必须移过的磁道的数目为 $222+18+769+182+217+99=1507$ 。

4. 为什么要引入设备独立性? 试说明如何实现设备独立性?

解: 在现代操作系统中, 为了提高系统的可适应性和可扩展性, 都毫无例外地实现了设备独立性, 也即设备无关性。其基本含义是, 应用程序独立于具体使用的物理设备, 即应用程序以逻辑设备名称来请求使用某类设备。进一步说, 在实现了设备独立性的功能后, 可带来两方面的好处: (1) 设备分配时的灵活性; (2) 易于实现 I/O 重定向(指用于 I/O 操作的设备可以更换即重定向, 而不必改变应用程序)。

为了实现设备的独立性, 应引入逻辑设备和物理设备两个概念。在应用程序中, 使用逻辑设备名称来请求使用某类设备; 而系统执行时, 是使用物理设备名称。鉴于驱动程序是一个与硬件(或设备)紧密相关的软件, 必须在驱动程序之上设置一层软件, 称为设备独立性软件, 以执行所有设备的公有操作、完成逻辑设备名到物理设备名的转换(为此应设置一张逻辑设备表)并向用户层(或文件层)软件提供统一接口, 从而实现设备的独立性。

5. 在设备分配中, 什么是安全分配方式和不安全分配方式? 请说明其特点

解:

① 所谓安全分配方式, 是指每当进程发出 I/O 请求后, 便进入阻塞状态, 直到其 I/O 操作完成时才被唤醒。在采用这种分配策略时, 一

一旦进程已经获得某种设备（资源）后便阻塞，使它不可能再请求任何资源，而在它运行时又不保持任何资源。因此，这种分配方式已经摒弃了造成死锁的四个必要条件之一的“请求和保持”条件，所以分配是安全的。其缺点是进程进展缓慢，即 CPU 与 I/O 设备是串行工作的。

② 所谓不安全分配方式，是指进程发出 I/O 请求后仍继续执行，需要时又可发出第二个 I/O 请求、第三个 I/O 请求。仅当进程所请求的设备已被另一个进程占有时，进程才进入阻塞状态。其优点是一个进程可同时操作多个设备，从而使进程推进迅速。而缺点是分配不安全，因为它可能具有“请求和保持”条件，所以可能造成死锁。因此，在设备分配程序中还需增加一个功能，用于对本次的设备分配是否会发生死锁进行安全性检查，仅当计算结果说明分配是安全的情况下才进行分配。

6. 【王道 2019, P273】

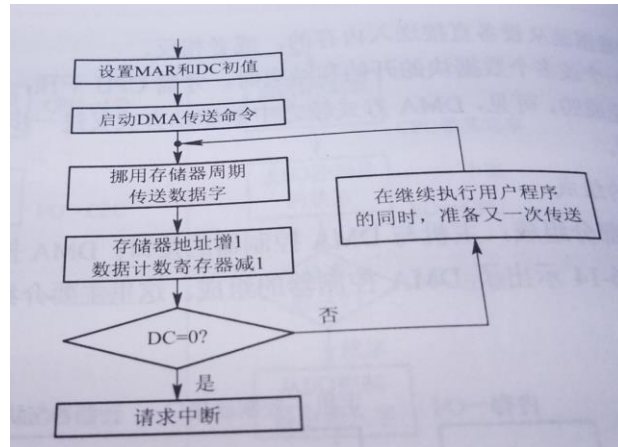
DMA 方式与中断控制方式的主要区别是什么？

DMA 控制方式与中断控制方式的主要区别为：

- 1) 中断控制方式在每个数据传送完成后中断 CPU，而 DMA 控制方式则是在所要求传送的一批数据全部传送结束时中断 CPU。
- 2) 中断控制方式的数据传送在中断处理时由 CPU 控制完成，而 DMA 控制方式则是在 DMA 控制器的控制下完成。不过，在 DMA 控制方式中，数据传送的方向、存放数据的内存始址及传送数据的长度等仍然由 CPU 控制。
- 3) DMA 方式以存储器为核心，中断控制方式以 CPU 为核心。因此 DMA 方式能与 CPU 并行工作。
- 4) DMA 方式传输批量的数据，中断控制方式传输则以字节为单位。

7. 试说明 DMA 的工作流程。

答：当 CPU 要从磁盘读入数据块时，先向磁盘控制器发送一条读命令。该命令被送到命令寄存器 CR 中。同时还发送本次要读入数据的内存起始目标地址，送入内存地址寄存器 MAR；本次要读数据的字节数送入数据计数器 DC，将磁盘中的源地址直接送 DMA 控制器的 I/O 控制逻辑上。然后启动 DMA 控制器传送数据，以后 CPU 便处理其它任务。整个数据传送过程由 DMA 控制器控制。下图为 DMA 方式的工作流程图。



8. 试说明设备驱动程序应具有哪些功能？

答：设备驱动程序的主要功能包括：

- (1) 将接收到的抽象要求转为具体要求；
- (2) 检查用户 I/O 请求合法性，了解 I/O 设备状态，传递有关参数，设置设备工作方式；
- (3) 发出 I/O 命令，启动分配到的 I/O 设备，完成指定 I/O 操作；
- (4) 及时响应由控制器或通道发来的中断请求，根据中断类型调用相应中断处理程序处理；
- (5) 对于有通道的计算机，驱动程序还应该根据用户 I/O 请求自动构成通道程序。

9. 设备中断处理程序通常需完成哪些工作？

答：设备中断处理程序通常需完成如下工作：

- (1) 唤醒被阻塞的驱动程序进程；
- (2) 保护被中断进程的 CPU 环境；
- (3) 分析中断原因、转入相应的设备中断处理程序；
- (4) 进行中断处理；
- (5) 恢复被中断进程。

10. 目前常用的磁盘调度算法有哪几种？每种算法优先考虑的问题是什么？

答：目前常用的磁盘调度算法有先来先服务、最短寻道时间优先及扫描等算法。

- (1) 先来先服务算法优先考虑进程请求访问磁盘的先后次序；
- (2) 最短寻道时间优先算法优先考虑要求访问的磁道与当前磁头所在磁道距离是否最近；
- (3) 扫描算法考虑欲访问的磁道与当前磁道间的距离，更优先考虑磁头当前的移动方向。

11. SP00Ling 系统由哪几部分组成？以打印机为例说明如何利用 SP00Ling 技术实现多个进程对打印机的共享？

答：组成：磁盘上的输入井和输出井，内存中的输入缓冲区和输出缓冲区，输入进程和输出进程。

对所有提出输出请求的用户进程，系统接受它们的请求时，并不真正把打印机分配给它们，而是由输出进程在输出井中为它申请一空闲缓冲区，并将要打印的数据卷入其中，输出进程再为用户进程申请一张空白的用户打印请求表，并将用户的打印请求填入表中，再将该表挂到打印机队列上。

这时，用户进程觉得它的打印过程已经完成，而不必等待真正的慢速的打印过程的完成。当打印机空闲时，输出进程将从请求队列队首取出一张打印请求表，根据表中的要求将要打印的数据从输出井传到内存输出缓冲区，再由打印机进行输出打印。打印完后，再处理打印队列中的一个打印请求表，实现了对打印机的共享。

12. 若干个等待访问磁盘者依次要访问的磁道为 20, 44, 40, 4, 80, 12, 76, 假设每移动一个磁道需要 3 毫秒时间，移动臂当前位于 40 号柱面，请按下列算法分别写出访问序列并计算为完成上述各次访问总共花费的寻道时间。（1）先来先服务算法；（2）最短寻道时间优先算法。（3）扫描算法（当前磁头移动的方向为磁道递增）

答：

（1）先来先服务算法磁道访问顺序为：20, 44, 40, 4, 80, 12, 76
寻道时间 = $(20+24+4+36+76+68+64) \times 3 = 292 \times 3 = 876$

（2）最短寻道时间优先算法磁道访问顺序为：40, 44, 20, 12, 4,

76, 80

寻道时间= $(0+4+24+8+8+72+4) * 3 = 120 * 3 = 360$

(3) 扫描算法磁道访问顺序为: 40, 44, 76, 80, 20, 12, 4

寻道时间= $(0+4+32+4+60+8+8) * 3 = 116 * 3 = 348$

六、文件管理

1. 有结构文件按记录的组织形式可分为哪些种类？试描述其主要特点【王道 2019, P212】

解：

1) 顺序文件。文件中的记录一个接一个地顺序排列，记录通常是定长的，可以顺序存储或以链表形式存储，在访问时需要顺序搜索文件。顺序文件有以下两种结构：

第一种是串结构，记录之间的顺序与关键字无关。通常的办法是由时间决定，即按存入时间的先后排列，最先存入的记录作为第 1 个记录，其次存入的为第 2 个记录，依此类推。

第二种是顺序结构，指文件中的所有记录按关键字顺序排列。

在对记录进行批量操作时，即每次要读或写一大批记录，对顺序文件的效率是所有逻辑文件中最高的；此外，也只有顺序文件才能存储在磁带上，并能有效地工作，但顺序文件对查找、修改、增加或删除单个记录的操作比较困难。

2) 索引文件。如图 4-1 所示。对于定长记录文件，如果要查找第 i 个记录，可直接根据下式计算来获得第 i 个记录相对于第一个记录的地址：

$$A_i = i \times L$$

然而，对于可变长记录的文件，要查找第 i 个记录时，必须顺序地查找前 $i-1$ 个记录，从而获得相应记录的长度 L ，才能按下式计算出第 i 个记录的首址：

$$A_i = \sum_{j=0}^{i-1} L_j + i$$

变长记录文件只能顺序查找，系统开销较大。为此，可以建立一张索引表以加快检索速度，索引表本身是定长记录的顺序文件。在记录很多或是访问要求高的文件中，需要引入索引以提供有效的访问。实际中，通过索引可以成百上千倍地提高访问速度。

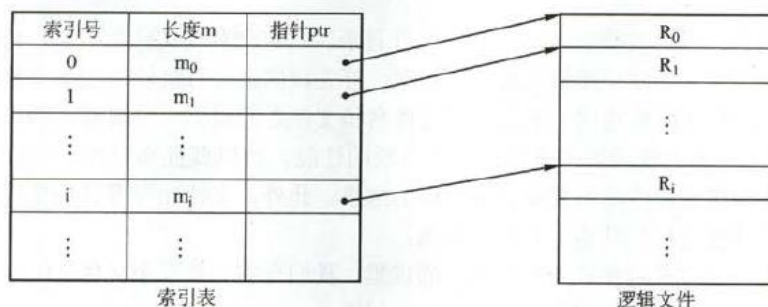


图 4-1 索引文件示意图

3) 索引顺序文件是顺序和索引两种组织形式的结合。索引顺序文件将顺序文件中的所有记录分为若干组，为顺序文件建立一张索引表，在索引表中为每组中的第一个记录建立一个索引项，其中含有该记录的关键字值和指向该记录的指针。

如图 4-2 所示，主文件名包含姓名和其他数据项。姓名为关键字，索引表中为每组的第一个记录（不是每个记录）的关键字值，用指针指向主文件中该记录的起始位置。索引表只包含关键字和指针两个数据项，所有姓名关键字递增排列。主文件中记录分组排列，同一个组中关键字可以无序，但组与组之间关键字必须有序。查找一个记录时，通过索引表找到其所在的组，然后在该组中使用顺序查找就能很快地找到记录。

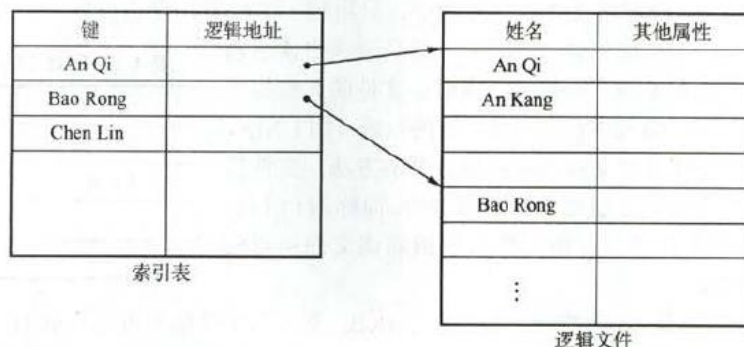


图 4-2 索引顺序文件示意图

对于含有 N 个记录的顺序文件，查找某关键字值的记录时平均需要查找 $N/2$ 次。在索引顺序文件中，假设 N 个记录分为 $\sqrt{N}$ 组，索引表中有 $\sqrt{N}$ 个表项，每组有 $\sqrt{N}$ 个记录，在查找某关键字值的记录时，先顺序查找索引表，需要查找 $\sqrt{N}/2$ 次，然后再在主文件中对应的组中顺序查找，也需要查找 $\sqrt{N}/2$ 次，这样总共查找 $\sqrt{N}/2 + \sqrt{N}/2 = \sqrt{N}$ 次。显然，索引顺序文件提高了查找效率，如果记录数很多，可以采用两级或多级索引。

索引文件和索引顺序文件都提高了存取的速度，但因为配置索引表而增加了存储空间。

4) 直接文件或散列文件 (Hash File): 给定记录的键值或通过 Hash 函数转换的键值直接决定记录的物理地址。这种映射结构不同于顺序文件或索引文件，没有顺序的特性。

散列文件有很高的存取速度，但是会引起冲突，即不同关键字的散列函数值相同。

2. 为文件分配磁盘块时，有哪些磁盘空间分配方式？试描述其主要特点

【王道 2019, P230】

1) 连续分配。连续分配方法要求每个文件在磁盘上占有一组连续的块, 如图 4-12 所示。磁盘地址定义了磁盘上的一个线性排序。这种排序使作业访问磁盘时需要的寻道数和寻道时间最小。

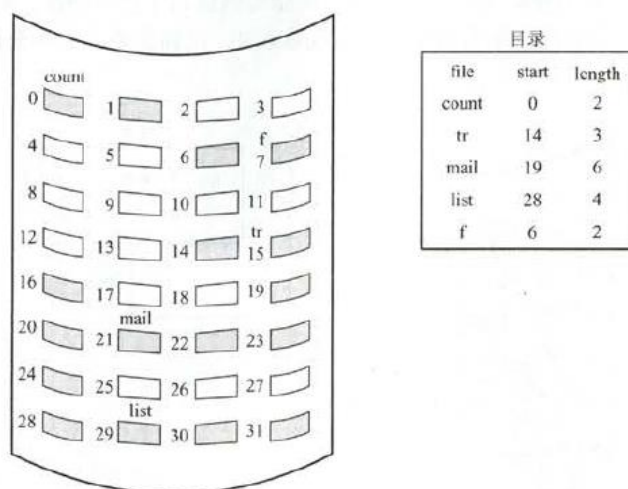


图 4-12 连续分配

文件的连续分配可以用第一块的磁盘地址和连续块的数量来定义。如果文件有 n 块长并从位置 b 开始, 那么该文件将占有块 $b, b+1, b+2, \dots, b+n-1$ 。一个文件的目录条目包括开始块的地址和该文件所分配区域的长度。

连续分配支持顺序访问和直接访问。其优点是实现简单、存取速度快。缺点在于, 文件长度不宜动态增加, 因为一个文件末尾后的盘块可能已经分配给其他文件, 一旦需要增加, 就需要大量移动盘块。此外, 反复增删文件后会产生外部碎片 (与内存管理分配方式中的碎片相似), 并且很难确定一个文件需要的空间大小, 因而只适用于长度固定的文件。

2) 链接分配。链接分配是采取离散分配的方式, 消除了外部碎片, 故而显著地提高了磁盘空间的利用率; 又因为是根据文件的当前需求, 为它分配必需的盘块, 当文件动态增长时, 可以动态地再为它分配盘块, 故而无需事先知道文件的大小。此外, 对文件的增、删、改也非常方便。链接分配又可以分为隐式链接和显式链接两种形式。

隐式连接如图 4-13 所示。每个文件对应一个磁盘块的链表; 磁盘块分布在磁盘的任何地方, 除最后一个盘块外, 每一个盘块都有指向下一个盘块的指针, 这些指针对用户是透明的。目录包括文件第一块的指针和最后一块的指针。

创建新文件时, 目录中增加一个新条目。每个目录项都有一个指向文件首块的指针。该指针初始化为 NULL 以表示空文件, 大小字段为 0。写文件会通过空闲空间管理系统找到空闲块, 将该块链接到文件的尾部, 以便写入。读文件则通过块到块的指针顺序读块。

隐式链接分配的缺点在于无法直接访问盘块, 只能通过指针顺序访问文件, 以及盘块指针消耗了一定的存储空间。隐式链接分配的稳定性也是一个问题, 系统在运行过程中由于软件或者硬件错误导致链表中的指针丢失或损坏, 会导致文件数据的丢失。

显式链接, 是指把用于链接文件各物理块的指针, 从每个物理块的块末尾中提取出来, 显式

Table, FAT)。

The diagram illustrates the implicit link allocation method. On the left is a directory table with 32 entries, each containing a file number and a pointer to the next block. On the right is a FAT (File Allocation Table) with columns for file, start, and end. The file 'jeep' is shown with a start block of 9 and an end block of 25. Arrows indicate the sequence of blocks allocated for the file 'jeep': 9, 10, 16, 25, 1, 18, 22, 27, and -1 (which points back to block 9). The directory table entries are as follows:

File No.	Next Block
0	
1	10
2	
3	
4	
5	
6	
7	
8	
9	16
10	25
11	
12	
13	
14	
15	
16	1
17	
18	
19	
20	
21	
22	
23	
24	
25	-1
26	
27	
28	
29	
30	
31	

目录

file	start	end
jeep	9	25

图 4-13 隐式链接分配

每个文件都有其索引块，这是一个磁盘块地址的数组。索引块的第 i 个条目指向文件的第 i 个块。目录条目包括索引块的地址。要读第 i 块，通过索引块的第 i 个条目的指针来查找和读入所需的块。

创建文件时，索引块的所有指针都设为空。当首次写入第 i 块时，先从空闲空间中取得一个块，再将其地址写到索引块的第 i 个条目。索引分配支持直接访问，且没有外部碎片问题。其缺点是由于索引块的分配，增加了系统存储空间的开销。索引块的大小是一个重要的问题，每个文件必须有一个索引块，因此索引块应尽可能小，但索引块太小就无法支持大文件。可以采用以下机制来处理这个问题。

链接方案：一个索引块通常为一个磁盘块，因此，它本身能直接读写。为了处理大文件，可以将多个索引块链接起来。

多层索引：多层索引使第一层索引块指向第二层的索引块，第二层索引块再指向文件块。这种方法根据最大文件大小的要求，可以继续到第三层或第四层。例如，4096B 的块，能在索引块中存入 1024 个 4B 的指针。两层索引允许 1048576 个数据块，即允许最大文件为 4GB。

混合索引：将多种索引分配方式相结合的分配方式。例如，系统既采用直接地址，又采用单级索引分配方式或两级索引分配方式。

表 4-2 是三种分配方式的比较。

表 4-2 文件三种分配方式的比较

	访问第 n 个记录	优 点	缺 点
顺序分配	需访问磁盘 1 次	顺序存取时速度快，当文件是定长时可以根据文件起始地址及记录长度进行随机访问	文件存储要求连续的存储空间，会产生碎片，也不利于文件的动态扩充
链接分配	需访问磁盘 n 次	可以解决外存的碎片问题，提高了外存空间的利用率，动态增长较方便	只能按照文件的指针链顺序访问，查找效率低，指针信息存放消耗外存空间
索引分配	m 级需访问磁盘 $m+1$ 次	可以随机访问，易于文件的增删	索引表增加存储空间的开销，索引表的查找策略对文件系统效率影响较大

此外，访问文件需要两次访问外存——首先要读取索引块的内容，然后再访问具体的磁盘块，因而降低了文件的存取速度。为了解决这一问题，通常将文件的索引块读入内存的缓冲区中，以加快文件的访问速度。

对于连续分配方式，优点是可以随机访问（磁盘），访问速度快；缺点是要求有连续的存储空间，容易产生碎片，降低磁盘空间利用率，并且不利于文件的增长扩充。

对于链接分配方式，优点是不要求连续的存储空间，更有效地利用磁盘空间，并且有利于扩充文件；缺点是只适合顺序访问，不适合随机访问；另外，链接指针占用一定空间，降低了存储效率，可靠性也差。

对于索引分配方式，优点是既支持顺序访问也支持随机访问，查找效率高，便于文件删除；缺点是索引表会占用一定的存储空间。

3. 什么是文件、文件系统？文件系统有哪些功能？

答：在计算机系统中，文件被解释为一组赋名的相关字符流的集合，或者是相关记录的集合。

文件系统是操作系统中与管理文件有关的软件和数据。

文件系统的功能是用户建立文件，撤销、读写修改和复制文件，以及完成对文件的按名存取和进行存取控制。

4. 什么是文件目录？文件目录中包含哪些信息？

答：一个文件的文件名和对该文件实施控制管理的说明信息称为该文件的说明信息，又称为该文件的目录。

文件目录中包含文件名、与文件名相对应的文件内部标识以及文件信息在文件存储设备上第一个物理块的地址等信息。另外还可能包含关于文件逻辑结构、物理结构、存取控制和管理等信息。

5. 6 文件存储空间管理中，对于空闲块的管理有三种常用方法，请说明其对空闲块是如何进行组织的？【王道 2019，P234】

答：

(1) 空闲表法

空闲表法属于连续分配方式，它与内存的动态分配方式类似，为每个文件分配一块连续的存储空间。系统为外存上的所有空闲区建立一张空闲盘块表，每个空闲区对应于一个空闲表项，其中包括表项序号、该空闲区第一个盘块号、该区的空闲盘块数等信息。再将所有空闲区按其起始盘块号递增的次序排列，见表 4-3。

表 4-3 空闲盘块表

序号	第一个空闲盘块号	空闲盘块数
1	2	4
2	9	3
3	15	5
4	—	—

空闲盘区的分配与内存的动态分配类似，同样是采用首次适应算法、循环首次适应算法等。例如，在系统为某新创建的文件分配空闲盘块时，先顺序地检索空闲盘块表的各表项，直至找到第一个其大小能满足要求的空闲区，再将该区分配给用户，同时修改空闲盘块表。

系统在对用户所释放的存储空间进行回收时，也采取类似于内存回收的方法，即要考虑回收区是否与空闲表中插入点的前区和后区相邻接，对相邻接者应予以合并。

(2) 空闲链表法

将所有空闲盘区拉成一条空闲链，根据构成链所用的基本元素不同，可把链表分成两种形式：空闲盘块链和空闲盘区链。

空闲盘块链是将磁盘上的所有空闲空间，以盘块为单位拉成一条链。当用户因创建文件而请求分配存储空间时，系统从链首开始，依次摘下适当的数目的空闲盘块分配给用户。当用户因删除文件而释放存储空间时，系统将回收的盘块依次插入空闲盘块链的末尾。这种方法的优点是分配和回收一个盘块的过程非常简单，但在为一个文件分配盘块时，可能要重复多次操作。

空闲盘区链是将磁盘上的所有空闲盘区（每个盘区可包含若干个盘块）拉成一条链。在每个盘区上除含有用于指示下一个空闲盘区的指针外，还应有能指明本盘区大小（盘块数）的信息。分配盘区的方法与内存的动态分区分配类似，通常采用首次适应算法。在回收盘区时，同样也要将回收区与相邻接的空闲盘区相合并。

(3) 位示图法

位示图是利用二进制的一位来表示磁盘中一个盘块的使用情况，磁盘上所有的盘块都有一个二进制位与之对应。当其值为“0”时，表示对应的盘块空闲；当其值为“1”时，表示对应的盘块已分配。位示图法示意如图 4-16 所示。

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	1	1	0	0	0	1	1	1	0	0	1	0	0	1	1	0
2	0	0	0	1	1	1	1	1	1	0	0	0	0	1	1	1
3	1	1	1	0	0	0	1	1	1	1	1	1	0	0	0	0
4																
⋮																
16																

图 4-16 位示图法示意

盘块的分配：

① 顺序扫描位示图，从中找出一个或一组其值为“0”的二进制位。

② 将所找到的一个或一组二进制位，转换成与之对应的盘块号。假定找到的其值为“0”的二进制位，位于位示图的第 i 行、第 j 列，则其相应的盘块号应按下式计算（ n 代表每行的位数）：

$$b=n(i-1)+j$$

③ 修改位示图，令 $\text{map}[i,j]=1$ 。

盘块的回收：

① 将回收盘块的盘块号转换成位示图中的行号和列号。

转换公式为

$$\begin{aligned} i &= (b-1) \text{DIV } n+1 \\ j &= (b-1) \text{MOD } n+1 \end{aligned}$$

② 修改位示图，令 $\text{map}[i,j]=0$ 。

空闲表：把所有空闲块组织成表

空闲链表法：把所有空闲块组织成链表

位示图：利用二进制的每位记录空闲块

6. 何谓数据项、记录 and 文件？

答：

① 数据项分为基本数据项和组合数据项。基本数据项描述一个对象某种属性的字符集，具有数据名、数据类型及数据值三个特性。组合数据项由若干数据项构成。

② 记录是一组相关数据项的集合，用于描述一个对象某方面的属性。

③ 文件是具有文件名的一组相关信息的集合。

7. 何谓逻辑文件？何谓物理文件？

答：逻辑文件是物理文件中存储的数据的一种视图方式，不包含具体数据，仅包含物理文件中数据的索引。物理文件又称文件存储结

构，是指文件在外存上的存储组织形式。

8. 何谓逻辑文件？何谓物理文件？

答：逻辑文件是物理文件中存储的数据的一种视图方式，不包含具体数据，仅包含物理文件中数据的索引。物理文件又称文件存储结构，是指文件在外存上的存储组织形式。

9. 何谓事务？如何保证事务的原子性？

答：事务是用于访问和修改各种数据项的一个程序单位。

要保证事务的原子性必须要求一个事务在对一批数据执行修改操作时，要么全部完成，用修改后的数据代替原来数据，要么一个也不改，保持原来数据的一致性。

七、操作系统接口

1. 操作系统包括哪几种类型的用户接口？它们分别适用于哪种情况？

答：操作系统包括四种类型的用户接口：命令接口（分为联机与脱机命令接口）、程序接口、图形化用户接口和网络用户接口。

命令接口和图形化用户接口支持用户直接通过终端来使用计算机系统，程序接口提供给用户在编制程序时使用，网络用户接口是面向网络应用的接口。

2. 试比较一般的过程调用和系统调用？

答：系统调用本质上是过程调用的一种特殊形式，与一般过程调用有差别：

（1）运行状态不同。一般过程调用的调用过程和被调用过程均为用户程序，或者均为系统程序，运行在同一系统状态（用户态或系统态）；系统调用的调用过程是用户态下的用户程序，被调用过程是系统态下的系统程序。

（2）软中断进入机制。一般的过程调用可直接由调用过程转向被调用过程；而系统调用不允许由调用过程直接转向被调用过程，一般通过软中断机制，先进入操作系统内核，经内核分析后，才能转向相应命令处理程序。

（3）返回及重新调度。一般过程调用在被调用结束后，返回调用点继续执行；系统调用被调用完后，要对系统中所有运行进程重新调度。只有当调用进程仍具有最高优先权才返回调用过程继续执行。

（4）嵌套调用。一般过程和系统调用都允许嵌套调用，注意系统过程嵌套而非用户过程。

3. 什么是系统调用？它都有哪些类型？

答：系统调用是指在操作系统内核设置的一组用于实现各种系统功能的子程序或过程，并提供给用户程序调用。主要类型包括：

（1）进程控制类。用于进程创建、终止、等待、替换、进程数据段大

小改变及进程标识符或指定进程属性获得等；

(2) 文件操纵类。用于文件创建、打开、关闭、读/写及文件读写指针移动和属性修改，目录创建及索引结点建立等；

(3) 进程通信类，用于实现通信机制如消息传递、共享存储区及信息量集机制等；

(4) 信息维护类，用于实现日期、时间及系统相关信息设置和获得。